



**Universidad de
Castilla-La Mancha**

VinOps:

**Sistema inteligente de apoyo a la decisión
en la industria vitivinícola**

Máster en Ingeniería Informática

Desarrollo e Integración de Servicios de IA

Integrantes del Grupo 9

Alejandro García Fernández

Pablo Alfonso López Fernández

Antonio Muñoz García

Pablo Parrilla Cañadas

Índice general

Introducción	8
Hito 1: Definición del problema y determinación del alcance	9
Contexto y definición del problema	9
Objetivos generales y específicos	10
Determinación del alcance	11
Planificación	11
Recursos Humanos: asignación de tareas y capacidad	11
Viabilidad. Estudio del ROI	11
Valor	12
Hito 2: Los datos – origen, descripción y análisis	14
Introducción al Hito 2	14
Fuentes, origen y tamaño	14
Justificación de la fuente de datos	14
Estructura y tamaño del dataset	15
Inventario de datos	15
Análisis de los datos	16
Estadísticos descriptivos	17
Análisis estadístico por clase	17
Análisis de distribuciones univariantes	19
Análisis de correlaciones	20
Análisis orientado a la detección de anomalías	20
Análisis de Componentes Principales (PCA)	21
Detección de valores atípicos	22
Exploración de los datos usando PCA	23

Sesgos y limitaciones del dataset	25
Sesgos detectados	25
Limitaciones para el modelado	26
Evaluación del rendimiento. Métricas	26
Formulación formal del problema	26
Métricas de evaluación propuestas	27
Rendimiento humano (HLP)	27
Benchmarks de referencia	28
Justificación del dataset y decisiones tomadas	28
Hito 3: Modelado, experimentación y validación	31
Introducción al Hito 3	31
Diseño experimental y metodología de validación	31
Selección de algoritmos y ajuste de hiperparámetros	32
Resultados y análisis comparativo	33
Análisis de errores	36
Explicabilidad y validación técnica	37
Hito 4: Despliegue y operacionalización del sistema	38
Introducción	38
Arquitectura general del despliegue	38
Decisiones de diseño y modularización	39
Contenerización del sistema	40
Orquestación con Docker Compose	40
Servicio de inferencia	41
Análisis de necesidades del sistema	42
Bonus: integración con MLflow	42
Evidencias del bonus MLflow	43
Valoración final del hito	44
Hito 5: Monitorización, feedback loop y shadow testing	46
Introducción	46
Arquitectura general del sistema de observabilidad	46
Instrumentación y trazabilidad del servicio	48
Detección de drift y sistema de alertas	50
Visualización con Prometheus y Grafana	53
Feedback loop y retraining semi-automático	54

Bonus: shadow testing y comparación champion-challenger	57
Decisiones de diseño globales	58
Limitaciones y valoración final del hito	59

Índice de tablas

1.	Características generales del dataset Wine UCI	15
2.	Distribución de clases en el dataset	15
3.	Inventario de datos – Diccionario de variables (parte 1/2)	16
4.	Inventario de datos – Diccionario de variables (parte 2/2)	16
5.	Estadísticos descriptivos del dataset	17
6.	Medias por clase y diferencia relativa entre clases	18
7.	Correlaciones relevantes entre variables del dataset	20
8.	Varianza explicada por las primeras componentes principales	21
9.	Outliers detectados mediante T^2 de Hotelling	22
10.	Outliers detectados mediante distancia al modelo (Q-residuals / SPE)	22
11.	Loadings de las variables en las cinco primeras componentes principales	23
12.	Métricas de evaluación propuestas para VinOps	27
13.	Benchmarks de referencia para el dataset Wine UCI [ACD91; ACD94; Jam+21; HTF09; Bre01; CV95; Fis36]	28
14.	Hiperparámetros optimizados y rangos de búsqueda	33
15.	Rendimiento medio estimado en validación cruzada (10-fold CV)	34
16.	Comparativa global de modelos sobre el conjunto de validación.	35
17.	Hiperparámetros óptimos seleccionados mediante Grid Search.	36
18.	Observaciones mal clasificadas comunes entre modelos	37
19.	Variables más relevantes según Random Forest	37
20.	Endpoints principales del sistema monitorizado	47
21.	Métricas operativas instrumentadas	49
22.	Métricas proxy del modelo	49
23.	Técnicas de detección de drift implementadas	51

24. Resumen del retraining con feedback acumulado	57
25. Decisiones de diseño principales del Hito 5	58

Índice de figuras

1.	Gráfico de scores para las dimensiones 1 y 2	24
2.	Gráfico de scores para las dimensiones 3 y 4	24
3.	Gráfico de individuos y contribución de las variables a las dos primeras dimensiones.	25
4.	Distribución del Accuracy en validación cruzada (10-fold CV) para los distintos modelos.	35
5.	Vista general del experimento registrado en MLflow.	43
6.	Detalle de la ejecución registrada, incluyendo métricas y parámetros.	44
7.	Modelo almacenado en MLflow y vinculado a la ejecución de origen.	44
8.	Entorno de observabilidad desplegado en Docker con la API monitorizada y los servicios auxiliares activos.	48
9.	Respuesta del endpoint /health del servicio principal en el puerto 8001.	48
10.	Ejemplo de predicción normal realizada por el endpoint /predict.	50
11.	Ejemplo de predicción anómala con indicadores activados en la respuesta del servicio.	50
12.	Alerta enviada por Telegram tras detectar un comportamiento anómalo en el sistema monitorizado.	52
13.	Notificación de deriva enviada al canal de Telegram del proyecto.	52
14.	Estado de drift del sistema tras la simulación, mostrando activación del monitor y recomendación de actuación.	53
15.	Vista de Prometheus con el target del servicio en estado UP.	53
16.	Dashboard de Grafana con paneles de latencia, memoria, confianza y distribución de clases.	54
17.	Registro de feedback manual enviado al sistema para alimentar el ciclo de mejora.	55
18.	Persistencia del feedback acumulado en el archivo CSV del sistema.	56
19.	Salida del endpoint /retrain mostrando la ejecución del reentrenamiento y sus resultados.	56
20.	Notificación emitida tras la ejecución del proceso de reentrenamiento.	56
21.	Recarga del nuevo modelo en memoria mediante /reload sin reiniciar el contenedor.	56

22.	Respuesta del endpoint /health del entorno bonus de shadow testing en el puerto 8002. . . .	57
23.	Predicción en modo champion-challenger con comparación entre ambos modelos.	58
24.	Estado agregado del shadow testing con métricas de comparación entre champion y challenger.	58
25.	Promoción del modelo challenger a champion mediante el endpoint /shadow/promote. . . .	58

La industria vitivinícola es uno de los sectores agroalimentarios de mayor impacto económico y cultural a nivel mundial. En España, con más de 2,6 millones de hectáreas de viñedo [Min23], representa uno de los pilares de la economía agraria y una seña de identidad nacional reconocida internacionalmente. La calidad del vino, entendida como la conformidad del producto con los estándares de su Denominación de Origen y las expectativas del consumidor, es el principal activo competitivo de cualquier bodega.

Sin embargo, garantizar esa calidad sigue siendo un desafío para el sector. Los métodos tradicionales de certificación, basados en análisis de laboratorio y en la evaluación sensorial de enólogos, son eficaces pero costosos, lentos y difíciles de escalar cuando el volumen de producción es elevado. Una bodega mediana puede necesitar evaluar miles de muestras por campaña, lo que supone una carga operativa y económica significativa.

Ante este escenario, la Inteligencia Artificial ofrece una oportunidad real para transformar la forma en que las bodegas gestionan la calidad de su producción. Este Hito 1 propone el diseño de un sistema inteligente capaz de clasificar la variedad de un vino a partir de sus propiedades fisicoquímicas, con el objetivo de apoyar al equipo técnico de la bodega en la toma de decisiones sobre la calidad de forma objetiva y reproducible.

Hito 1: Definición del problema y determinación del alcance

En este capítulo se detalla la definición del problema para el proyecto de la asignatura Desarrollo e Integración de Servicios de IA, así como la determinación de su alcance.

Contexto y definición del problema

En la industria vitivinícola, la certificación de la calidad y autenticidad del vino son procesos de gran relevancia económica, legal y reputacional. La correcta identificación de la variedad de uva y la detección de irregularidades en la composición del vino son requisitos indispensables para garantizar el cumplimiento de las normativas de las Denominaciones de Origen, proteger al consumidor frente a fraudes de etiquetado y mantener la competitividad de la bodega en un mercado cada vez más exigente.

Actualmente, este proceso depende casi en su totalidad de dos métodos complementarios: los análisis físico-químicos de laboratorio y la cata sensorial realizada por enólogos expertos. Aunque ambos métodos ofrecen resultados de calidad, presentan limitaciones evidentes cuando se aplican a gran escala. Por un lado, los análisis de laboratorio son precisos pero lentos y costosos. Por otro, la cata sensorial, aunque valiosa, introduce subjetividad y depende de la disponibilidad de personal especializado, lo que dificulta su aplicación sistemática en el día a día de una bodega con alta producción.

Además, las características del vino varían de una cosecha a otra en función del clima, el suelo, las técnicas de cultivo y los procesos de fermentación. Esto significa que los criterios de evaluación deben adaptarse continuamente, algo que los procedimientos manuales actuales no están diseñados para hacer de forma ágil y automatizada.

El problema que aborda este proyecto tiene dos componentes complementarios. El **componente principal** es la clasificación supervisada de la variedad vinícola: dado un análisis fisicoquímico de una muestra, el sistema debe identificar a cuál de los tres cultivares pertenece. Este componente se abordará directamente con el dataset UCI Wine Recognition, que proporciona etiquetas de clase para las 178 muestras disponibles.

El **componente secundario** es la detección de anomalías en nuevos lotes de producción. Dado que las características del vino varían con cada cosecha, el sistema debe ser capaz de identificar muestras cuya composición fisicoquímica se aleja significativamente de los patrones aprendidos durante el entrenamiento, alertando al equipo técnico antes de que el modelo realice una clasificación poco fiable. Este componente se implementará analizando las distribuciones estadísticas de las variables del dataset y definiendo

umbrales de desviación a partir de los datos de entrenamiento.

En definitiva, no se trata solamente de predecir la calidad de un vino, sino de construir un servicio inteligente de apoyo a la decisión que acompañe al enólogo en su trabajo diario, aportando consistencia, transparencia y capacidad de respuesta ante los cambios propios del sector.

La fuente de datos seleccionada es el *Wine Recognition Dataset*, disponible en el repositorio UCI Machine Learning Repository [ACD91]. Recoge análisis fisicoquímicos de vinos italianos de tres cultivares distintos, descritos mediante 13 propiedades químicas. La variable objetivo es la **clase de cultivo**, que toma tres valores posibles correspondientes a las tres variedades vinícolas presentes en el dataset.

Objetivos generales y específicos

El problema puede formularse con la siguiente pregunta: *¿Cómo diseñar un sistema de inteligencia artificial capaz de identificar la variedad de un vino y detectar anomalías en su composición de forma objetiva, consistente y mantenible a lo largo del tiempo, apoyando la toma de decisiones del equipo técnico de una bodega?*

Para dar respuesta a esto, se establece como **objetivo general** del proyecto diseñar, implementar y desplegar un sistema de inteligencia artificial que permita a una bodega auditar la calidad y autenticidad de su producción vinícola de forma automática, objetiva y continuada, garantizando su utilidad real en un entorno de producción.

Los **objetivos específicos** que permitirán alcanzar este objetivo general son:

- Formalizar el concepto de calidad y autenticidad vinícola desde un punto de vista cuantitativo, identificando qué propiedades fisicoquímicas son relevantes para la diferenciación de variedades.
- Seleccionar y procesar los datos disponibles sobre la composición química del vino, asegurando su calidad y fiabilidad como base del sistema.
- Implementar y comparar distintos modelos de inteligencia artificial capaces de identificar la variedad de un vino a partir de su composición, seleccionando el más adecuado para su uso en producción.
- Evaluar los modelos tanto con métricas técnicas como con criterios alineados con las necesidades reales del sector vitivinícola.
- Diseñar el sistema como un producto integrable en el flujo de trabajo de una bodega, mantenible a lo largo del tiempo y capaz de adaptarse a los cambios de cada nueva cosecha.
- Validar el sistema con datos representativos de la producción vinícola real, garantizando su utilidad práctica para el personal técnico de la bodega.

Determinación del alcance

En esta sección se determina el alcance del proyecto, considerando los componentes clave que definen su viabilidad y su potencial como producto real.

Planificación

El proyecto se desarrollará a lo largo de cinco hitos iterativos que cubren el ciclo de vida completo de un proyecto de Machine Learning, desde la definición del problema hasta su despliegue.

Recursos Humanos: asignación de tareas y capacidad

El desarrollo del sistema VinOps se llevará a cabo por un equipo de cuatro integrantes con roles complementarios:

- **Antonio Muñoz García — Experto en dominio:** valida que los resultados del sistema sean coherentes con el conocimiento experto del sector vitivinícola.
- **Alejandro García Fernández — Analista de datos:** responsable de la recopilación y preparación de los datos fisicoquímicos del vino.
- **Pablo Parrilla Cañadas — Ingeniero de Machine Learning:** diseña, entrena y evalúa los modelos de clasificación.
- **Pablo Alfonso López Fernández — Ingeniero de despliegue:** garantiza que el sistema funcione de forma estable y pueda actualizarse ante nuevas cosechas.

El **cliente** es una bodega o cooperativa vitivinícola, cuyo director técnico actúa como *stakeholder* principal, validando los resultados y orientando la evolución del sistema.

Viabilidad. Estudio del ROI

La viabilidad del proyecto se evalúa considerando tanto su impacto técnico como económico. Para la estimación del ROI se parten de los siguientes supuestos orientativos:

- **Duración del proyecto:** 3 meses.
- **Equipo técnico:** 4 personas.
- **Dedicación estimada:** 10 horas semanales por persona.
- **Coste estimado por hora:** 35 euros, valor coherente con los salarios medios de perfiles técnicos especializados en datos e inteligencia artificial en España [Gla24].

Los valores anteriores son orientativos y podrían variar en función del contexto real del proyecto y del perfil del equipo técnico contratado.

$$\text{Coste total} = 4 \times 10 \text{ h/semana} \times 12 \text{ semanas} \times 35 \text{ €/hora} = 16.800 \text{ €}$$

En cuanto a los beneficios, según datos del sector vitivinícola español [Obs22], una bodega mediana realiza aproximadamente 2.000 análisis de calidad por campaña a un coste estimado de 15 euros por análisis. Automatizando el 70 % de ellos, el ahorro anual sería de 21.000 euros. Considerando además la reutilización del sistema en campañas sucesivas, el ROI estimado es el siguiente:

$$\text{ROI} = \frac{21.000 - 16.800}{16.800} \approx 25 \%$$

Este resultado indica que el proyecto es económicamente viable desde el primer año, con un retorno creciente en campañas posteriores.

Valor

El valor aportado por el sistema VinOps se manifiesta en distintas dimensiones: productiva, social, organizativa y tecnológica.

Valor productivo. Desde el punto de vista operativo, VinOps aporta a la bodega un mecanismo objetivo y sistemático de clasificación de variedades vinícolas y detección de anomalías en la composición del producto. Este enfoque permite complementar el criterio del enólogo con una base cuantitativa sólida y reproducible, contribuyendo a identificar de forma consistente desviaciones en la calidad antes de que el producto llegue al mercado y a garantizar que los criterios de evaluación se mantengan estables a lo largo del tiempo, independientemente del personal disponible.

Valor social y reputacional. La certificación objetiva y transparente de la variedad y calidad del vino refuerza la credibilidad de la bodega ante consumidores, distribuidores y organismos reguladores. En un mercado donde la trazabilidad y la autenticidad son cada vez más valoradas, disponer de un sistema de auditoría inteligente puede convertirse en un argumento diferenciador frente a la competencia y en una garantía de cumplimiento normativo ante las Denominaciones de Origen.

Valor organizativo. El sistema VinOps constituye un activo estratégico reutilizable campaña tras campaña, extensible a nuevas líneas de producto y potencialmente integrable en los procesos internos de la bodega. Su diseño como producto, y no como experimento puntual, garantiza que la inversión realizada genere valor de forma continuada y que el sistema pueda evolucionar junto con las necesidades de la organización.

Valor tecnológico. Desde una perspectiva científico-técnica, el proyecto propone una metodología reproducible para la evaluación automatizada de la calidad vinícola, validada sobre datos reales de competición procedentes del repositorio UCI Machine Learning Repository [ACD91]. La combinación de modelos

interpretables con criterios de evaluación alineados con el dominio vitivinícola sitúa a VinOps no solo como una herramienta de predicción, sino como una plataforma de inteligencia aplicada al sector agroalimentario, con capacidad de adaptación y mantenimiento a largo plazo.

Hito 2: Los datos – origen, descripción y análisis

Introducción al Hito 2

En este hito se describen y analizan los datos disponibles para abordar el problema planteado en el Hito anterior: diseñar, implementar y validar un sistema inteligente capaz de **clasificar la variedad vinícola de un vino** a partir de sus propiedades fisicoquímicas, con el objetivo de apoyar al enólogo en la toma de decisiones de calidad de forma objetiva y reproducible.

El análisis que se describe tiene como objetivo comprender los datos, detectar posibles sesgos y limitaciones, y evaluar en qué medida el conjunto de datos disponible permite abordar el problema planteado.

Fuentes, origen y tamaño

La fuente de datos seleccionada es el **Wine Recognition Dataset**, disponible en el repositorio UCI Machine Learning Repository [ACD91]. Este dataset recoge los resultados de análisis químicos realizados sobre vinos producidos en la misma región de Italia, pero procedentes de **tres cultivares distintos**.

Justificación de la fuente de datos

La elección de este dataset se debe a los siguientes motivos:

- **Relevancia para el dominio:** Las 13 variables fisicoquímicas (alcohol, acidez, fenoles, etc.) son las magnitudes que los laboratorios vitivinícolas miden de forma rutinaria para la certificación de calidad y variedad.
- **Variables bien definidas:** Cada columna tiene una interpretación clara desde la perspectiva enológica, esto facilita la validación con el experto en dominio.
- **Disponibilidad y reproducibilidad:** Al ser un dataset público y ampliamente utilizado, permite comparar los resultados del sistema con benchmarks establecidos.
- **Ausencia de valores faltantes:** El dataset está completo, lo que elimina la necesidad de estrategias de imputación que pudieran introducir sesgos adicionales.

Estructura y tamaño del dataset

El dataset presenta las siguientes características generales:

Tabla 1: Características generales del dataset Wine UCI

Característica	Valor
Número de instancias (filas)	178
Número de variables (columnas)	13 + 1 variable objetivo
Valores faltantes	0 (dataset completo)
Tipo de problema	Clasificación multiclase supervisada
Número de clases	3 (cultivares de vino)
Tipo de datos	Numérico continuo (todas las variables)
Formato	CSV (.csv)

La distribución de instancias por clase es la siguiente:

Tabla 2: Distribución de clases en el dataset

Clase	Cultivar	N.º muestras	%
1	Cultivar 1	59	33.1 %
2	Cultivar 2	71	39.9 %
3	Cultivar 3	48	27.0 %
Total	–	178	100 %

Desde el punto de vista del volumen, nos encontramos ante un problema de **small data** (178 instancias). Dado el reducido número de instancias, el uso de modelos con alta capacidad expresiva debe controlarse mediante validación cruzada estratificada para evitar sobreajuste.

Adicionalmente, la distribución de clases no es perfectamente uniforme (27%–40%), por lo que nos encontramos con una distribución de clases levemente desbalanceada, pero no es extrema ni problemática. A pesar de no ser un requisito indispensable dado el bajo desbalanceo, se utilizarán métricas que no sean sensibles a la disparidad de clases (F1-Macro o Precisión Balanceada) para asegurar un rendimiento equilibrado entre los diferentes tipos de vino.

Inventario de datos

En esta sección se detalla el diccionario de variables del dataset. Este inventario ha sido elaborado con el apoyo del experto en dominio del equipo (Antonio Muñoz García), quien ha validado la relevancia enológica de cada variable con respecto al problema de clasificación de cultivares.

Tabla 3: Inventario de datos – Diccionario de variables (parte 1/2)

Variable	Descripción	Tipo	Unidad
Alcohol	Contenido alcohólico del vino	Num. cont.	% vol
MalicAcid	Concentración de ácido málico	Num. cont.	g/L
Ash	Contenido en cenizas (minerales totales)	Num. cont.	g/L
AlcalinityAsh	Alcalinidad de las cenizas	Num. cont.	mEq/L
Magnesium	Concentración de magnesio	Num. cont.	mg/L
TotalPhenols	Contenido total de fenoles	Num. cont.	g/L
Flavanoids	Contenido de flavanoides	Num. cont.	g/L

Tabla 4: Inventario de datos – Diccionario de variables (parte 2/2)

Variable	Descripción	Tipo	Unidad
NonflavanoidPhenols	Fenoles no flavanoides	Num. cont.	g/L
Proanthocyanins	Contenido de proantocianidinas	Num. cont.	g/L
ColorIntensity	Intensidad del color del vino	Num. cont.	u.a.
Hue	Tonalidad del vino	Num. cont.	u.a.
OD280/OD315	Relación de absorbancias (calidad proteínas)	Num. cont.	u.a.
Proline	Concentración del aminoácido prolina	Num. cont.	mg/L
Class	Clase de cultivar (1, 2 o 3) – variable objetivo	Categórica	–

Todas las variables de entrada son numéricas continuas, lo que simplifica el preprocesado. Sin embargo, la diversidad de escalas (e.g., Proline oscila entre 278 y 1680 mg/L mientras que Hue toma valores entre 0.48 u.a. y 1.71 u.a.) hace imprescindible aplicar una **normalización** antes del entrenamiento de modelos sensibles a la escala de los datos.

Análisis de los datos

El análisis exploratorio de los datos se ha realizado mediante un cuaderno de R desarrollado específicamente para este hito [**eda_r_script**], cuyo código está disponible en el repositorio VinOps [Gar+26a].

1. Estadísticos descriptivos de todas las variables.
2. Análisis estadístico por clase.
3. Análisis de distribuciones por clase.
4. Análisis de correlaciones entre variables.

5. Análisis de componentes principales (PCA).
6. Detección de valores atípicos mediante PCA
7. Exploración de los datos usando PCA

Estadísticos descriptivos

La Tabla 5 resume los principales estadísticos descriptivos del dataset.

Tabla 5: Estadísticos descriptivos del dataset

Variable	Mín	Q1	Med.	Media	Q3	Máy
Alcohol	11.03	12.36	13.05	13.00	13.68	14.83
MalicAcid	0.74	1.60	1.87	2.34	3.08	5.80
Ash	1.36	2.21	2.36	2.37	2.56	3.23
AlcalinityAsh	10.6	17.2	19.5	19.5	21.5	30.0
Magnesium	70	88	98	99.7	107	162
TotalPhenols	0.98	1.74	2.35	2.29	2.80	3.88
Flavanoids	0.34	1.21	2.13	2.03	2.88	5.08
NonflavanoidPhenols	0.13	0.27	0.34	0.36	0.44	0.66
Proanthocyanins	0.41	1.25	1.55	1.59	1.95	3.58
ColorIntensity	1.28	3.22	4.69	5.06	6.20	13.00
Hue	0.48	0.78	0.96	0.96	1.12	1.71
OD280/OD315	1.27	1.94	2.78	2.61	3.17	4.00
Proline	278	500	673	747	985	1680

De la inspección de estos estadísticos se extraen las siguientes observaciones:

- **Diversidad de escalas:** La variable Proline presenta valores hasta 1680 mg/L, mientras que Hue oscila entre 0.48 y 1.71. Esta heterogeneidad hace imprescindible la normalización antes del entrenamiento de modelos basados en distancias o gradientes.
- **Distribuciones asimétricas:** MalicAcid presenta una media (2.34) notablemente superior a su mediana (1.87), indicando sesgo positivo. Lo mismo ocurre con ColorIntensity y Proline. Estas asimetrías pueden impactar en modelos que asumen normalidad.
- **Variables con baja variabilidad relativa:** Ash y NonflavanoidPhenols presentan rangos intercuartílicos reducidos, lo que sugiere que tal vez no puedan aportar tanta información por sí solas.

Análisis estadístico por clase

Para cuantificar de forma descriptiva el grado de separación entre clases en cada variable, se empleó la siguiente medida de diferencia relativa entre medias:

$$\Delta_{rel} = \frac{\max(\bar{x}_k) - \min(\bar{x}_k)}{\frac{1}{K} \sum_{k=1}^K \bar{x}_k}$$

donde:

- $\bar{x}_k$ es la media de la variable en la clase k ,
- K es el número de clases (en nuestro caso, $K = 3$).

Esta medida calcula el rango máximo entre medias de clase y lo normaliza dividiéndolo por la media global de dichas medias, obteniendo una cantidad global que permite determinar la diferenciación entre las clases.

Tabla 6: Medias por clase y diferencia relativa entre clases

Variable	Clase 1	Clase 2	Clase 3	Δ_{rel}	Nivel
Flavanoids	2.98	2.08	0.78	1.13	Muy alta
Color_Intensity	5.53	3.09	7.40	0.81	Muy alta
Proline	1115.71	519.51	629.90	0.79	Muy alta
OD280_OD315	3.16	2.79	1.68	0.58	Alta
Malic_Acid	2.01	1.93	3.33	0.58	Alta
Total_Phenols	2.84	2.26	1.68	0.51	Alta
Proanthocyanins	1.90	1.63	1.15	0.48	Moderada
Nonflavanoid_Phenols	0.29	0.36	0.45	0.43	Moderada
Hue	1.06	1.06	0.68	0.41	Moderada
Alcalinity_of_Ash	17.04	20.24	21.42	0.22	Bajo
Magnesium	106.34	94.55	99.31	0.12	Bajo
Alcohol	13.74	12.28	13.15	0.11	Bajo
Ash	2.46	2.24	2.44	0.09	Bajo

De este análisis descriptivo se observa que:

- Flavanoids, Color_Intensity y Proline presentan las mayores diferencias relativas entre medias de clase, lo que indica una mayor diferenciación entre cultivares en términos de valores medios.
- En contraste, variables como Ash, Alcohol, Magnesium y Alcalinity_of_Ash muestran medias más similares entre clases.
- La **Clase 3** destaca por un perfil medio claramente diferenciado en varias variables, especialmente por sus valores bajos en Flavanoids y altos en Color_Intensity y Malic_Acid.

- Las **Clases 1 y 2** presentan medias más próximas en varias variables, lo que podría hacer más compleja su separación si se consideran únicamente diferencias de promedio.

Cabe señalar que esta medida describe únicamente diferencias entre medias y no tiene en cuenta la dispersión dentro de cada clase, por lo que sus resultados deben interpretarse como un análisis preliminar.

Análisis de distribuciones univariantes

El análisis de los histogramas generados para cada variable permite caracterizar la forma, dispersión y posibles valores extremos presentes en el dataset. Este estudio descriptivo constituye una etapa previa al análisis detallado y no implica, por sí mismo, conclusiones sobre capacidad discriminante.

- **Flavanoids:** La distribución global presenta una estructura compatible con bimodalidad, con concentraciones de valores aproximadamente en los intervalos 0.5–1.0 g/L y 2.5–3.0 g/L. Esta configuración sugiere la posible existencia de subgrupos diferenciados en la muestra. No obstante, la interpretación en términos de separación entre clases requiere un análisis más profundo.
- **Proline:** Variable con marcada asimetría positiva y amplio rango de valores (hasta 1680 mg/L). La presencia de cola larga y valores extremos indica alta variabilidad y posible influencia de outliers. Dada su escala significativamente mayor respecto a otras variables, resulta imprescindible considerar estandarización previa al modelado.
- **MalicAcid:** Distribución asimétrica hacia la derecha, con presencia de valores elevados en la cola superior. Esta estructura sugiere heterogeneidad en los niveles de acidez málica dentro de la muestra.
- **ColorIntensity:** Presenta cola larga hacia la derecha, con valores superiores a 10 que podrían considerarse extremos en términos descriptivos. La dispersión observada indica variabilidad relevante en esta característica físico-química.
- **Ash, AlcalinityAsh, Magnesium:** Distribuciones aproximadamente simétricas y concentradas en torno a sus valores centrales, con menor dispersión relativa en comparación con variables como **Proline** o **ColorIntensity**.

En conjunto, el análisis univariante revela que:

- Existen diferencias notables en escala entre variables.
- Varias características presentan asimetría y posibles valores extremos.
- La heterogeneidad observada sugiere la conveniencia de aplicar técnicas de normalización o estandarización antes del entrenamiento de modelos.

Cabe destacar que estas observaciones describen únicamente el comportamiento marginal de cada variable ($P(X)$). La evaluación de su capacidad para diferenciar entre cultivares requiere el análisis de las distribuciones condicionadas por clase ($P(X | \text{Clase})$).

Análisis de correlaciones

El heatmap de correlaciones de Pearson muestra la existencia de relaciones lineales relevantes entre varias variables físico-químicas. Las correlaciones de mayor magnitud ($|r| \geq 0.50$) se resumen a continuación:

Tabla 7: Correlaciones relevantes entre variables del dataset

Par de variables	r	Interpretación
TotalPhenols – Flavanoids	0.86	Muy alta positiva
Flavanoids – OD280_OD315	0.79	Alta positiva
TotalPhenols – OD280_OD315	0.70	Alta positiva
Flavanoids – Proanthocyanins	0.65	Moderada-alta positiva
Alcohol – Proline	0.64	Moderada-alta positiva
TotalPhenols – Color_Intensity	0.61	Moderada positiva
Hue – OD280_OD315	0.57	Moderada positiva
Malic_Acid – Hue	-0.56	Moderada negativa
Flavanoids – Nonflavanoid_Phenols	-0.54	Moderada negativa
Color_Intensity – Hue	-0.52	Moderada negativa

Se observa un bloque claramente correlacionado formado por TotalPhenols, Flavanoids y OD280_OD315, con coeficientes elevados y consistentes. Esta estructura sugiere que estas variables capturan información química parcialmente solapada, posiblemente asociada al contenido fenólico del vino. En términos estadísticos, ello indica la existencia de redundancia informativa y cierto grado de multicolinealidad entre estas características.

Asimismo, Proanthocyanins y Color_Intensity muestran asociaciones relevantes con este grupo, lo que refuerza la idea de que existe un conjunto de variables relacionadas entre sí que describen propiedades químicas conectadas.

Estas relaciones indican la presencia de estructura lineal interna en el dataset y podrían justificar la exploración de técnicas de reducción dimensional en la siguiente sección.

Análisis orientado a la detección de anomalías

Además de la clasificación supervisada de cultivares, el sistema VinOps tiene como segundo objetivo detectar muestras anómalas en nuevos lotes de producción: vinos cuya composición fisicoquímica se aleja

de los patrones conocidos, lo que podría indicar errores de medición, contaminación, fraude de etiquetado o variaciones extremas de cosecha.

Para abordar este componente del problema, el análisis de los datos se ha complementado con el estudio de los siguientes factores:

Distribución estadística de cada variable por clase. Las Tablas 5 y 6 permiten caracterizar el rango normal de operación de cada variable para cada cultivar. Una muestra nueva cuyas variables se sitúen fuera del intervalo $[\bar{x}_k - 3\sigma_k, \bar{x}_k + 3\sigma_k]$ para la clase predicha será marcada como potencialmente anómala. Aplicando este criterio sobre el dataset, se detectan pocas observaciones fuera de los umbrales, en su mayoría por valores altos. Variables como *Magnesium*, *Total Phenols* y *Proanthocyanins* son las que más veces aparecen fuera de rango, y hay casos donde una misma muestra falla en varias variables a la vez (por ejemplo, la observación 122), lo que indica que es claramente atípica. Las observaciones detectadas son recogidas para la posterior examinación por parte del enólogo.

Outliers detectados en el modelo PCA. Las observaciones identificadas en las Tablas 9 y 10 mediante T^2 de Hotelling y Q-residuals constituyen la referencia empírica del comportamiento anómalo en el espacio multidimensional. El sistema de detección de anomalías se calibrará usando estos mismos umbrales estadísticos (F_{95} , F_{99}), de forma que una muestra nueva que supere el umbral F_{99} en el espacio PCA sea marcada automáticamente para revisión por el enólogo.

Variables más sensibles a la detección de anomalías. Del análisis de loadings PCA (Tabla 11) se desprende que las variables con mayor carga en las primeras componentes (Flavanoids, TotalPhenols, OD280/OD315 en PC1; ColorIntensity y Proline en PC2) son las más informativas tanto para la clasificación como para la detección de desviaciones. Sin embargo, para la detección de anomalías cobran especial relevancia las variables con menor variabilidad intraclase, como Ash, AlcalinityAsh y Hue, ya que cualquier desviación significativa en estas variables es más indicativa de un problema real que de variabilidad natural entre muestras del mismo cultivar.

Análisis de Componentes Principales (PCA)

Se ha aplicado PCA sobre el dataset (con variables estandarizadas) para obtener una representación de baja dimensionalidad que permita visualizar estructura y posibles agrupamientos por clase.

Tabla 8: Varianza explicada por las primeras componentes principales

Componente	Varianza explicada	Varianza acumulada
PC1	36.2 %	36.2 %
PC2	19.2 %	55.4 %
PC3	11.12 %	66.52 %
PC4	7.07 %	73.6 %
PC5	6.57 %	80.15 %

Las conclusiones del análisis PCA son:

- En el plano PC1–PC2 se observa una estructura agrupada por clases, con una diferenciación apreciable entre los tres cultivares. No obstante, existe cierto solapamiento, especialmente en la región ocupada por la Clase 2, por lo que no puede afirmarse una separabilidad perfecta en esta proyección bidimensional.
- **PC1** está fuertemente influida por variables relacionadas con el contenido fenólico (Flavanoids, TotalPhenols, OD280/OD315), lo que sugiere que esta componente recoge un eje asociado a la riqueza polifenólica del vino.
- **PC2** está principalmente determinada por ColorIntensity, MalicAcid y Proline, aportando una segunda dimensión relevante de variabilidad química.
- Las dos primeras componentes explican el 58.4 % de la varianza total, lo que permite una representación visual informativa, aunque parte de la estructura del dataset permanece en componentes posteriores, pudiendo llegar al 75 % de la varianza utilizando dos componentes principales más.

Detección de valores atípicos

Se han aplicado dos técnicas complementarias:

1. **Análisis univariante (boxplot IQR):** Se identifican valores fuera del rango $[Q1 - 1,5 \cdot IQR, Q3 + 1,5 \cdot IQR]$ para cada variable.
2. **Análisis multivariante (T^2 de Hotelling y Q-residuals/SPE):** Implementado sobre el espacio PCA para detectar observaciones anómalas en el espacio multidimensional reducido.

Tabla 9: Outliers detectados mediante T^2 de Hotelling

Observación	Nivel de anomalía
26, 61, 74, 124	Moderado (entre F_{95} y F_{99})
60, 70, 96, 97, 111, 122, 125	Extremo (por encima de F_{99})

Tabla 10: Outliers detectados mediante distancia al modelo (Q-residuals / SPE)

Observación	Nivel de anomalía
72, 79, 100, 111, 122	Moderado (entre Q_{95} y Q_{99})
96, 159, 160	Extremo (por encima de Q_{99})

Estas observaciones serán revisadas junto con el experto en dominio para determinar si corresponden a errores de medición en laboratorio o a vinos genuinamente atípicos. La decisión de eliminarlos o mantenerlos se tomará en el hito de preprocesado. También se observa que algunas observaciones detectadas

como atípicas mediante PCA también aparecen al aplicar el criterio de $\pm 3\sigma$, como es el caso de las observaciones 26, 70, 96, 111, 122 y 124, lo que refuerza su carácter anómalo al aparecer en ambos análisis y la necesidad de que el experto enólogo las revise.

Exploración de los datos usando PCA

Tabla 11: Loadings de las variables en las cinco primeras componentes principales

Variable	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
Alcohol	0.1443	0.4837	-0.2074	-0.0179	0.2657
Malic_Acid	-0.2452	0.2249	0.0890	0.5369	-0.0352
Ash	-0.0021	0.3161	0.6262	-0.2142	0.1430
Alcalinity_of_Ash	-0.2393	-0.0106	0.6121	0.0609	-0.0661
Magnesium	0.1420	0.2996	0.1308	-0.3518	-0.7270
Total_Phenols	0.3947	0.0650	0.1462	0.1981	0.1493
Flavanoids	0.4229	-0.0034	0.1507	0.1523	0.1090
Nonflavanoid_Phenols	-0.2985	0.0288	0.1704	-0.2033	0.5007
Proanthocyanins	0.3134	0.0393	0.1495	0.3991	-0.1369
Color_Intensity	-0.0886	0.5300	-0.1373	0.0659	0.0764
Hue	0.2967	-0.2792	0.0852	-0.4278	0.1736
OD280_OD315	0.3762	-0.1645	0.1660	0.1841	0.1012
Proline	0.2868	0.3649	-0.1267	-0.2321	0.1579

Se puede observar que la primera dimensión está fuertemente influida por variables relacionadas con el contenido fenólico (Flavonoides, Total de Fenoles y OD280/OD315), lo que sugiere que esta componente recoge un eje asociado a la riqueza polifenólica del vino.

La segunda dimensión está principalmente determinada por la Intensidad del color, Ácido Málico y prolina, de manera que la segunda dimensión para aportar cierta relevancia química.

La tercera dimensión parece tener principalmente influencia por parte de las cenizas del vino y su alcalinidad, mientras que en la cuarta solamente destacan el Ácido Málico y un poco los proantocianidinas

Finalmente, la quinta dimensión está principalmente influida por el Magnesio, con una carga negativa elevada, y en menor medida por los Fenoles no flavonoides, lo que indica que esta componente recoge variaciones asociadas principalmente al contenido mineral del vino.

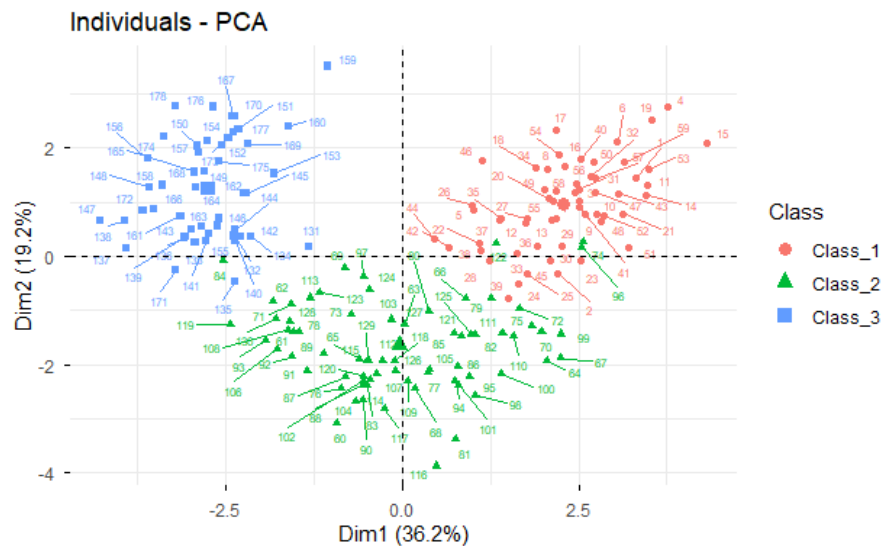


Figura 1: Gráfico de scores para las dimensiones 1 y 2

En el primer gráfico de scores (Figura 1), se puede observar cómo, solamente con dos dimensiones, todos los individuos por clase se pueden ver bastante bien separados. Esto es un buen indicativo de cara al modelado y la predicción de sus clases. Se puede ver cómo la clase 2 puede encontrarse algo solapada con las clases 1 y 3, pero cómo estas dos últimas sí están completamente diferenciadas entre ellas.

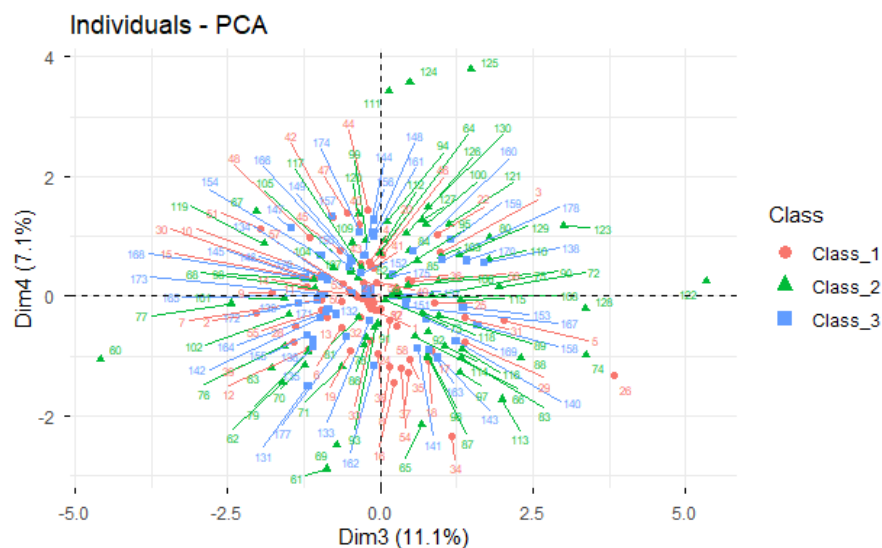


Figura 2: Gráfico de scores para las dimensiones 3 y 4

Sin embargo, en la visualización de la tercera y cuarta dimensión (Figura 2) es complicado extraer conclusiones, ya que se observan todos los individuos entremezclados y no es posible distinguir una separación clara entre las clases.

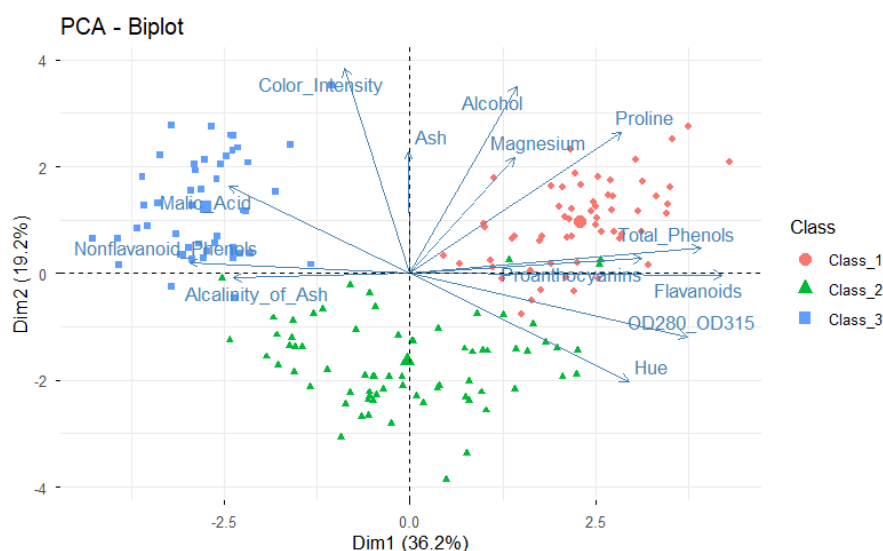


Figura 3: Gráfico de individuos y contribución de las variables a las dos primeras dimensiones.

En el biplot mostrado en la Figura 3 se observa que la Clase 1 se concentra principalmente en la zona positiva de la primera dimensión, alineada con las direcciones de *Flavonoides*, *Fenoles Totales*, *OD280/OD315* y *Prolina*, lo que sugiere que estos vinos presentan valores relativamente altos en dichas variables. Por el contrario, la Clase 3 se sitúa mayoritariamente en la región negativa de la primera componente y positiva de la segunda, en la dirección de *Ácido Málico* y *Fenoles No Flavonoides*, indicando una mayor presencia relativa de estas características y valores más bajos en los compuestos fenólicos asociados a la Dimensión 1, que caracterizan a la Clase 1.

La Clase 2 aparece principalmente en la parte inferior del gráfico, asociada a valores negativos de la segunda componente, en oposición a variables como *Intensidad del Color*, *Cenizas* y *Alcohol*. Además, algunas observaciones de esta clase se ven ligeramente influenciadas por variables como *Alcalinidad de las Cenizas*, que aparecen próximas a su región. En conjunto, la separación entre clases se explica principalmente por el gradiente de compuestos fenólicos en la primera dimensión y por variables relacionadas con el color, las cenizas y el alcohol en la segunda.

Sesgos y limitaciones del dataset

Sesgos detectados

Sesgo geográfico.

Todos los vinos proceden de la misma región de Italia. El clasificador puede no generalizar a vinos de otras denominaciones de origen (p.ej., Rioja, Burdeos).

Sesgo temporal

El dataset no incluye el año de cosecha. Las propiedades fisicoquímicas varían entre campañas

por factores climáticos y de producción, por lo que el modelo puede degradarse al aplicarse a vinos de cosechas muy diferentes.

Leve desbalanceo de clases.

La Clase 3 (27%) está subrepresentada respecto a la Clase 2 (40%), lo que puede producir métricas de *accuracy* que deben ser revisadas.

Ausencia de variables sensoriales.

El dataset recoge únicamente propiedades fisicoquímicas. La evaluación real del enólogo también incorpora aspectos organolépticos (aroma, sabor, textura) no representados en los datos.

Limitaciones para el modelado

Small data (178 instancias).

El reducido tamaño incrementa el riesgo de sobreajuste en modelos complejos. Se recomienda **validación cruzada** (k-fold con $k = 10$).

Multicolinealidad entre variables fenólicas.

La alta correlación entre TotalPhenols, Flavanoids y OD280/OD315 puede causar inestabilidad numérica en modelos como la regresión logística sin regularización.

Escalas heterogéneas.

Requiere normalización obligatoria antes de modelos basados en distancias (KNN, SVM) o en gradientes (redes neuronales, regresión logística).

Evaluación del rendimiento. Métricas

Formulación formal del problema

El sistema VinOps debe aprender una función:

$$f: \mathbb{R}^{13} \rightarrow \{1, 2, 3\}$$

tal que, dado un vector de características fisicoquímicas $\mathbf{x} = (x_1, x_2, \dots, x_{13})^\top$, el sistema prediga correctamente la clase de cultivar $y \in \{1, 2, 3\}$.

Se trata de un **problema de clasificación multiclase supervisada** con 3 clases no ordenadas, lo que lo diferencia del problema de *ranking* abordado en el ejemplo HARP del profesor.

Métricas de evaluación propuestas

Tabla 12: Métricas de evaluación propuestas para VinOps

Métrica	Justificación
Macro F1-Score	Media no ponderada del F1 por clase. Penaliza por igual los errores en clases minoritarias. Recomendada para datasets con desbalanceo.
Accuracy	Proporción de muestras correctamente clasificadas. Útil como métrica complementaria pero sensible al desbalanceo.
Balanced Accuracy	Media de la sensibilidad (recall) por clase. Robusta ante desbalanceo.
Matthews Correlation Coefficient	Coefficiente de correlación entre predicciones y etiquetas reales. Considera todas las celdas de la matriz de confusión y es robusto ante desbalanceo.
Cohen's Kappa	Mide el acuerdo más allá del azar. Útil para comparar con HLP.

Rendimiento humano (HLP)

Aunque no existe literatura específica que cuantifique el rendimiento humano (Human Level Performance) para este dataset, es razonable esperar que un enólogo experto clasificando manualmente a partir de 13 valores fisicoquímicos obtenga una precisión significativamente inferior (estimada entre 75 % y 85 %). Esta estimación se basa en la complejidad cognitiva de procesar simultáneamente 13 variables numéricas, frente a la capacidad de los algoritmos para capturar patrones multivariantes complejos.

La literatura científica reporta que modelos de ML bien entrenados alcanzan *accuracy* del 98–99 % con técnicas como LDA o SVM. El sistema VinOps debería superar claramente esta referencia humana.

Los valores anteriores sobre el rendimiento humano son orientativos y podrían variar en función del contexto real del proyecto y del perfil del equipo técnico contratado.

Benchmarks de referencia

Tabla 13: Benchmarks de referencia para el dataset Wine UCI [ACD91; ACD94; Jam+21; HTF09; Bre01; CV95; Fis36]

Modelo	Accuracy aprox.	Observaciones
LDA (Análisis Discriminante Lineal)	$\approx 99\%$	Alta interpretabilidad
SVM (kernel RBF)	$\approx 98\%$	Requiere normalización
Random Forest	$\approx 98\%$	Robusto a outliers
Regresión Logística (L2)	$\approx 97\%$	Requiere normalización
KNN ($k = 3$)	$\approx 96\%$	Requiere normalización
Árbol de decisión	$\approx 92\%$	Alta interpretabilidad

Estos valores servirán como referencia para evaluar el rendimiento de los modelos que se desarrollarán en el Hito 3. Un modelo que no supere los benchmarks de la Tabla 13 deberá ser revisado antes de proceder a las fases de despliegue.

Justificación del dataset y decisiones tomadas

A lo largo de este análisis se han tomado las siguientes decisiones:

- Uso del dataset Wine UCI sin modificaciones en este hito:** El dataset no presenta valores faltantes ni errores evidentes. La limpieza de outliers se abordará en el Hito 3 (preprocesado).
- Inclusión de las 13 variables en el análisis:** Aunque variables como Ash presentan bajo poder discriminante, se mantienen en el inventario para que la fase de selección de características del Hito 3 determine formalmente cuáles deben eliminarse.
- Selección del Macro F1-Score como métrica principal:** Justificada por el desbalanceo moderado entre clases y por la necesidad de que el sistema funcione bien en todas las variedades vitícolas, incluyendo la menos representada (Clase 3).
- No aplicación de pipelines de datos en este hito:** A diferencia del ejemplo HARP, el dataset Wine UCI no requiere la integración de múltiples fuentes ni la transformación de datos brutos. Los datos ya están estructurados en formato CSV, por lo que la construcción de un pipeline complejo no aporta valor en esta fase.
- No aplicación de *data augmentation* ni integración de fuentes adicionales:** El reducido tamaño del dataset (178 instancias) podría en principio justificar el uso de técnicas para ampliar artificialmente el volumen de datos o la búsqueda de fuentes complementarias. Ambas alternativas han sido valoradas explícitamente y descartadas por las siguientes razones:

- **Fuentes de datos externas:** Con el objetivo de ampliar el volumen de datos disponibles, se investigó la posibilidad de integrar el dataset *Wine Quality* del repositorio UCI [CEB09], que cuenta con 4.898 instancias de vinos tintos y blancos portugueses (*Vinho Verde*) con 11 variables físico-químicas medidas en laboratorio. Sin embargo, tras un análisis comparativo detallado, se descartó su integración por las siguientes incompatibilidades estructurales:
 - **Variables distintas:** Wine Quality recoge `fixed_acidity`, `volatile_acidity`, `citric_acid`, `residual_sugar`, `chlorides`, `free_sulfur_dioxide`, `total_sulfur_dioxide`, `density`, `pH` y `sulphates`, ninguna de las cuales coincide directamente con las 13 variables del dataset Wine Recognition UCI (fenoles, flavanoides, prolina, etc.).
 - **Variable objetivo incompatible:** Wine Quality etiqueta cada muestra con una puntuación de calidad ordinal entre 0 y 10 (variable continua), mientras que Wine Recognition UCI etiqueta por **cultivar** (variable categórica con 3 clases). Ambas tareas son fundamentalmente distintas y no pueden combinarse en un mismo modelo supervisado.
 - **Origen geográfico diferente:** Wine Quality recoge vinos portugueses (*Vinho Verde*), mientras que Wine Recognition UCI recoge vinos italianos de una misma región. Su integración ampliaría el sesgo geográfico en lugar de reducirlo.
 - **Ausencia de identificador común para la combinación de datos:** Incluso en un hipotético escenario en el que las variables y la variable objetivo fueran compatibles, la integración de ambos datasets sería técnicamente inviable. El dataset Wine Recognition UCI no incluye ningún identificador único de muestra (código de lote, número de análisis o similar) que permita establecer una correspondencia con registros de otras fuentes. Sin una clave común de enlace, cualquier intento de combinación entre datasets produciría un emparejamiento arbitrario sin validez científica.
- **Data augmentation sintética (SMOTE):** Técnicas como SMOTE [Cha+02] generan muestras sintéticas interpolando entre observaciones reales de la misma clase y son especialmente útiles cuando el desbalanceo entre clases es severo o cuando el tamaño muestral es insuficiente para aprender las fronteras de decisión. En nuestro caso, el desbalanceo es leve (27%–40%) y, como se verá posteriormente, el análisis PCA muestra que las tres clases parecen linealmente separables en el espacio reducido. Aplicar SMOTE en un espacio de alta separabilidad puede generar puntos sintéticos en regiones de solapamiento que introduzcan ruido y degraden la calidad del clasificador [HTF09]. Por tanto, la técnica no aporta valor en este contexto específico.
- **Suficiencia del dataset para el problema planteado:** Considerando que el dataset no presenta valores faltantes, que las clases son separables según el análisis exploratorio y que se aplicará validación cruzada estratificada con $k = 10$ particiones para maximizar el aprovechamiento de las 178 muestras disponibles, se estima que el volumen de datos es suficiente para entrenar y

validar los modelos previstos en el Hito 3 sin necesidad de datos sintéticos adicionales.

Cabe señalar que esta decisión es específica del contexto actual del proyecto. En un escenario real donde el sistema VinOps se extienda a nuevas denominaciones de origen con menor volumen de datos etiquetados, tanto la recolección activa de muestras de laboratorio como el uso de *data augmentation* serían estrategias a reconsiderar.

Hito 3: Modelado, experimentación y validación

Introducción al Hito 3

Tras haber comprendido la naturaleza fisicoquímica de los datos y su estructura multivariante en el Hito 2, el presente capítulo aborda la fase central del sistema VinOps: el **modelado predictivo**. El objetivo principal es diseñar, entrenar y validar un conjunto de algoritmos de *Machine Learning* capaces de clasificar de forma robusta la variedad de los cultivares de vino.

Para garantizar la viabilidad del sistema en un entorno de producción real, este proceso no se limita a la búsqueda de la máxima exactitud, sino que sigue una metodología científica estricta: preprocesamiento fundamentado, diseño de experimentos controlados mediante ajuste de hiperparámetros (*hyperparameter tuning*), evaluación mediante métricas robustas al desbalanceo y, finalmente, un análisis de interpretabilidad para validar las reglas subyacentes del modelo.

Diseño experimental y metodología de validación

En esta fase se ha abordado el problema de clasificación multiclase sobre el *Wine Recognition Dataset* mediante un diseño experimental orientado a maximizar la capacidad de generalización de los modelos. Para ello, se ha realizado una partición estratificada de los datos, destinando el 70% de las observaciones al entrenamiento y el 30% al test, preservando en ambos subconjuntos la proporción original de las tres clases de vino.

Sobre el conjunto de entrenamiento se ha aplicado una validación cruzada de 10 particiones (*10-fold Cross-Validation*) durante el ajuste de los modelos. Esta estrategia permite evitar la necesidad de reservar un conjunto de validación independiente, algo especialmente conveniente en escenarios de *small data*, donde resulta prioritario aprovechar al máximo la información disponible. En consecuencia, el conjunto de test queda completamente aislado y se utiliza exclusivamente para la evaluación final.

Siguiendo los criterios definidos en el Hito 2, la comparación entre modelos se ha basado principalmente en el Macro F1-Score y la Balanced Accuracy, complementadas por *Accuracy*, *Kappa* y el coeficiente *Matthews Correlation Coefficient* (MCC). Este conjunto de métricas permite evaluar el rendimiento de forma robusta en un contexto multiclase con ligero desbalanceo.

Selección de algoritmos y ajuste de hiperparámetros

La selección de algoritmos responde a una estrategia en dos niveles. Por un lado, se incluyen los modelos presentes en los benchmarks de referencia identificados en el Hito 2 (LDA, Regresión Logística Multinomial, SVM radial, kNN y Árbol de Decisión), cuya inclusión permite contrastar directamente los resultados obtenidos con los reportados en la literatura. Por otro lado, el equipo propone **Random Forest como modelo principal del sistema VinOps**, por las siguientes razones técnicas:

- **Robustez ante multicolinealidad:** El análisis de correlaciones del Hito 2 reveló un bloque fenólico altamente correlacionado (TotalPhenols, Flavanoids, OD280/OD315, $|r| \geq 0,70$). Random Forest es inherentemente robusto ante multicolinealidad gracias a la selección aleatoria de variables en cada división (mtry), a diferencia de modelos lineales como la regresión logística que pueden verse afectados por inestabilidad numérica.
- **No requiere normalización:** A diferencia de SVM y kNN, Random Forest no es sensible a la escala de las variables. Dado que el dataset presenta escalas muy heterogéneas (Proline: 278–1680 mg/L; Hue: 0,48–1,71 u.a.), esta propiedad reduce el riesgo de introducir sesgos durante el preprocesado.
- **Interpretabilidad mediante importancia de variables:** El sistema VinOps necesita ofrecer al enólogo una explicación de qué variables han determinado la clasificación. Random Forest proporciona de forma nativa una medida de importancia de variables basada en la reducción del criterio de impureza (índice de Gini), directamente interpretable desde el dominio vitivinícola.
- **Resistencia al sobreajuste en *small data*:** Al promediar sobre múltiples árboles entrenados en subconjuntos aleatorios (*bagging*), Random Forest reduce la varianza del estimador, mitigando el riesgo de sobreajuste que el reducido tamaño muestral hace especialmente crítico [Bre01].

El resto de modelos se incluyen como referencias experimentales que permiten cuantificar la ganancia obtenida frente a alternativas más simples y contextualizar el rendimiento de Random Forest en el espectro de la literatura.

Se han entrenado y evaluado siete modelos representativos, abarcando enfoques lineales, probabilísticos, basados en distancias y en árboles de decisión.

- **LDA (Análisis Discriminante Lineal):** Modelo lineal que proyecta los datos maximizando la separación entre clases bajo supuestos de normalidad.
- **Regresión Logística Multinomial:** Extensión multiclase que modela directamente la probabilidad de pertenencia a cada clase.
- **Naive Bayes (NB):** Clasificador probabilístico basado en independencia condicional entre variables.

- **kNN**: Modelo basado en instancias que clasifica según la proximidad a los vecinos más cercanos.
- **SVM radial**: Modelo que construye fronteras no lineales mediante un kernel radial.
- **Árbol (CART)**: Modelo jerárquico basado en reglas binarias, altamente interpretable.
- **Random Forest (RF)**: Ensamblado de árboles entrenados sobre subconjuntos aleatorios, que reduce la varianza y mejora la robustez.

El ajuste de hiperparámetros se ha realizado mediante **Grid Search** dentro del esquema de validación cruzada. Esta elección se justifica por el reducido tamaño del dataset y por el carácter acotado de los espacios de búsqueda, lo que permite una exploración exhaustiva sin recurrir a métodos más complejos.

Las rejillas de búsqueda se definieron seleccionando únicamente los hiperparámetros con mayor impacto en el rendimiento y acotando su rango para evitar una complejidad innecesaria.

En **Naive Bayes**, se exploró `usekernel`, que determina si la densidad se estima mediante una distribución gaussiana o mediante métodos no paramétricos, relevante ante posibles asimetrías.

Para **kNN**, se ajustó el número de vecinos k , que controla el equilibrio sesgo-varianza. Se evaluaron valores en $k \in \{2, 3, 5, 8, 10\}$.

En **SVM radial**, se optimizaron C (penalización de errores) y σ (radio de influencia), explorando valores bajos y medios para capturar distintos niveles de complejidad.

Para el **árbol (CART)**, se ajustó `cp`, que controla la poda, considerando valores entre 0.01 y 0.10.

En **Random Forest**, se optimizó `mtry`, número de variables por división, evaluando $\{2, 4, 6, 8, 10\}$ alrededor de la referencia $\sqrt{p}$.

LDA y la regresión multinomial no requieren ajuste explícito de hiperparámetros, por lo que se emplean como modelos de referencia de menor complejidad.

Tabla 14: Hiperparámetros optimizados y rangos de búsqueda

Modelo	Hiperparámetros (rango)
Naive Bayes	<code>usekernel</code> $\in \{TRUE, FALSE\}$
kNN	$k \in \{2, 3, 5, 8, 10\}$
SVM radial	$C \in \{0, 1, 10, 100\}$, $\sigma \in \{0, 01, 0, 05, 0, 1\}$
Árbol (CART)	$cp \in [0, 01, 0, 10]$
Random Forest	<code>mtry</code> $\in \{2, 4, 6, 8, 10\}$

Resultados y análisis comparativo

El modelado, la experimentación y la validación se han realizado mediante un cuaderno de R desarrollado específicamente para este hito [**model_r_script**], cuyo código está disponible en el repositorio VinOps [Gar+26a].

Antes de evaluar el rendimiento final sobre el conjunto de test, se analiza el comportamiento de los modelos durante el entrenamiento mediante validación cruzada (10-fold CV). Esta evaluación permite estimar el rendimiento esperado y estudiar tanto la capacidad de aprendizaje como la estabilidad de los modelos.

Tabla 15: Rendimiento medio estimado en validación cruzada (10-fold CV)

Modelo	F1_CV	Bal. Acc. CV	Accuracy_CV	Kappa_CV	MCC_CV
Multinomial	1.0000	1.0000	1.0000	1.0000	1.0000
SVM radial	0.9918	0.9949	0.9921	0.9880	0.9881
LDA	0.9844	0.9895	0.9841	0.9760	0.9762
Random Forest	0.9771	0.9834	0.9762	0.9639	0.9640
Naive Bayes	0.9690	0.9774	0.9683	0.9519	0.9522
kNN	0.9540	0.9684	0.9524	0.9281	0.9306
Árbol	0.8852	0.9104	0.8810	0.8188	0.8191

Como se observa en la Tabla 15, la mayoría de modelos alcanzan un rendimiento muy elevado durante la validación cruzada, destacando la regresión multinomial, que presenta métricas perfectas, seguida de SVM radial y LDA con resultados prácticamente óptimos. Este comportamiento sugiere que el problema presenta una estructura altamente separable en el espacio de características, donde distintas familias algorítmicas son capaces de capturar eficazmente la frontera de decisión.

No obstante, estos resultados deben interpretarse con cautela, ya que un rendimiento perfecto en validación cruzada puede ser indicativo de un modelo altamente ajustado al conjunto de entrenamiento. Por este motivo, será necesario contrastar estos resultados con el comportamiento observado en el conjunto de test, donde se evaluará la capacidad real de generalización.

El análisis de la estabilidad del aprendizaje (Figura 4) muestra que la mayoría de modelos presentan una dispersión reducida en la métrica de Accuracy. Aunque métricas como el F1-score o la Balanced Accuracy serían más coherentes con el enfoque multiclase adoptado, el análisis de la variabilidad en validación cruzada se ha realizado sobre Accuracy debido a las limitaciones del paquete `caret`, que únicamente proporciona de forma nativa esta métrica (junto con Kappa) en los objetos de tipo `resamples`. La extensión de este análisis a métricas personalizadas requeriría un procesamiento adicional fuera del flujo estándar de la librería.

Random Forest destaca por combinar un alto rendimiento con una variabilidad reducida, lo que refuerza su robustez como modelo candidato. Por el contrario, modelos como kNN y, especialmente, el árbol de decisión presentan una mayor dispersión, reflejando una mayor sensibilidad a la partición de los datos y, por tanto, una menor estabilidad. De manera general, los resultados evidencian un aprendizaje estable en la mayoría de modelos, si bien será necesario evaluar su comportamiento en el conjunto de test para confirmar su capacidad de generalización y descartar posibles efectos de sobreajuste.

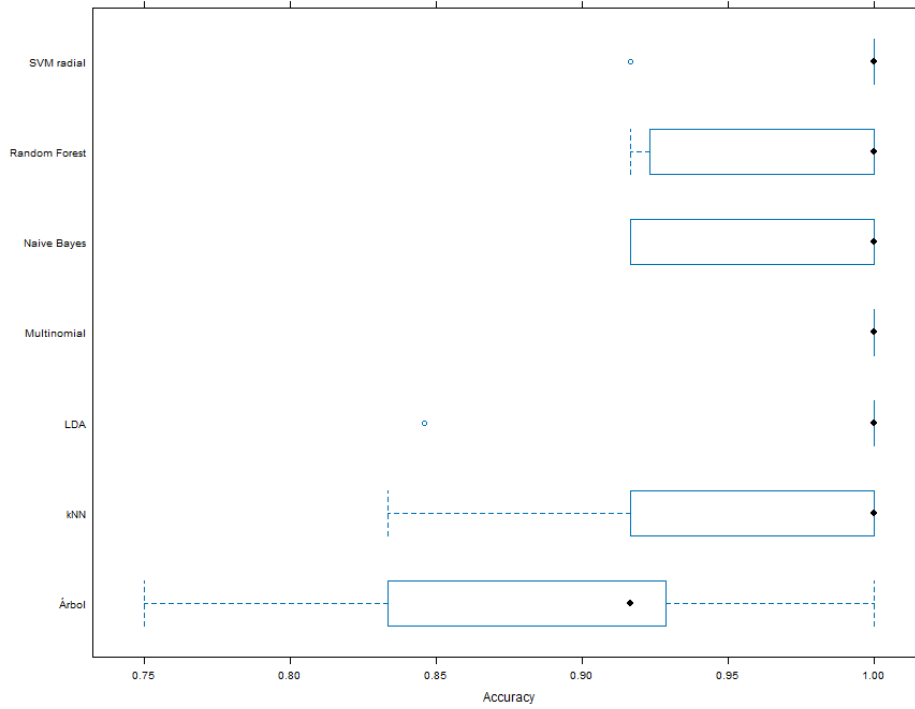


Figura 4: Distribución del Accuracy en validación cruzada (10-fold CV) para los distintos modelos.

Tras la fase de entrenamiento y ajuste, los modelos fueron evaluados sobre el conjunto de prueba. A continuación, se presenta la tabla comparativa con los algoritmos, ordenados por su rendimiento en la métrica principal (F1-Score).

Tabla 16: Comparativa global de modelos sobre el conjunto de validación.

Modelo	F1-Score	Balanced Acc.	Accuracy	Kappa	MCC
Random Forest	0.9804	0.9877	0.9808	0.9709	0.9714
Naive Bayes	0.9804	0.9877	0.9808	0.9709	0.9714
LDA	0.9623	0.9750	0.9615	0.9419	0.9435
Multinomial	0.9611	0.9754	0.9615	0.9420	0.9441
Árbol	0.9444	0.9573	0.9423	0.9122	0.9138
SVM radial	0.9421	0.9630	0.9423	0.9133	0.9179
kNN	0.9231	0.9503	0.9231	0.8846	0.8911
Aleatorio - Baseline	0.4290	0.5732	0.4231	0.1329	0.1335

Como se observa en la Tabla 16, los resultados avalan la viabilidad técnica del sistema. La comparativa de modelos muestra un rendimiento muy elevado en la mayoría de algoritmos, destacando Naive Bayes y Random Forest como los modelos con mejores resultados, alcanzando métricas prácticamente idénticas. Esto sugiere que el problema presenta una estructura bien definida en el espacio de características, donde tanto enfoques probabilísticos como modelos basados en ensamblado son capaces de capturar eficazmente las relaciones entre variables.

LDA y la regresión multinomial presentan un rendimiento muy cercano, lo que refuerza la idea de

que una parte importante de la separación entre clases puede modelarse de forma aproximadamente lineal. Por su parte, modelos como SVM radial y kNN no aportan mejoras significativas, lo que indica que la complejidad adicional no resulta determinante en este caso.

El árbol de decisión obtiene un rendimiento inferior, coherente con su mayor varianza en datasets pequeños. Finalmente, el baseline aleatorio se sitúa claramente por debajo del resto, confirmando que los modelos entrenados están capturando una estructura real en los datos.

Aunque Naive Bayes y Random Forest alcanzan un rendimiento prácticamente idéntico, se selecciona Random Forest como modelo final del sistema por su mayor robustez ante relaciones no lineales, su menor dependencia de supuestos distribucionales fuertes y su capacidad de ofrecer medidas de importancia de variables interpretables para el experto enológico.

Tabla 17: Hiperparámetros óptimos seleccionados mediante Grid Search.

Modelo	Hiperparámetros óptimos
Naive Bayes	{fL: 0, usekernel: FALSE, adjust: 1}
Árbol	{cp: 0.07}
Random Forest	{mtry: 2}
kNN	{kmax: 10, distance: 2, kernel: optimal}
SVM radial	{sigma: 0.01, C: 10}

En la Tabla 17 hiperparámetros seleccionados muestran configuraciones relativamente simples, lo que refuerza la idea de que el problema no requiere modelos altamente complejos para alcanzar un buen rendimiento. En particular, destaca el bajo valor de *mtry* en Random Forest y el valor reducido de σ en SVM, indicando fronteras de decisión relativamente suaves y bien definidas.

Análisis de errores

Tal como se observa en la Tabla 18, el análisis de errores revela que las observaciones mal clasificadas se concentran principalmente en la clase 2, que en varios casos es predicha como clase 3. Este patrón sugiere la existencia de una región de solapamiento entre ambas clases, donde la separación no es completamente nítida.

En particular, la observación 62 destaca como un caso crítico, ya que es sistemáticamente mal clasificada por todos los modelos. Este comportamiento indica que no se trata de una limitación específica de un algoritmo, sino de una ambigüedad intrínseca en los datos.

La comparación de la observación 62 con las medias de las clases 2 y 3 confirma que presenta un comportamiento mixto, con ciertas variables más próximas a la clase 3, lo que explica su confusión. En este sentido, se trata de un punto situado en la frontera de decisión, más que de un error aleatorio.

Tabla 18: Observaciones mal clasificadas comunes entre modelos

Obs	Clase Real	Predicción
62	2	3
69	2	3
119	2	3

Por otro lado, los outliers detectados previamente mediante PCA no parecen estar directamente relacionados con los errores, ya que únicamente uno de ellos aparece en el conjunto de test y además se clasifica correctamente. No obstante, este aspecto debería analizarse con mayor profundidad en escenarios más amplios, dado que su impacto podría manifestarse en otras particiones o conjuntos de datos.

Explicabilidad y validación técnica

El análisis de importancia de variables mediante Random Forest permite identificar qué características fisicoquímicas tienen mayor influencia en la clasificación. Como se observa, las variables más relevantes son la prolina, la intensidad del color, los flavonoides y el ratio OD280/OD315, seguidas por el alcohol y los compuestos fenólicos totales.

Tabla 19: Variables más relevantes según Random Forest

Variable	Importancia
Proline	100.00
Color_Intensity	95.01
OD280_OD315	84.07
Flavanoids	75.11
Alcohol	72.78

En la Tabla 19, se muestra la importancia de las variables más relevantes según el modelo Random Forest. Estas variables están directamente relacionadas con la composición fenólica y las características organolépticas del vino, lo que resulta coherente con el dominio enológico. En particular, la prolina y la intensidad del color son conocidos indicadores de diferenciación entre variedades, mientras que los flavonoides y el ratio OD280/OD315 reflejan la concentración de compuestos fenólicos, altamente discriminativos.

En contraste, variables como el *ash* o los *nonflavanoid phenols* presentan una contribución prácticamente nula, lo que indica que aportan poca información relevante para la tarea de clasificación.

Este patrón es consistente con el análisis PCA realizado previamente, donde estas mismas variables mostraban una elevada contribución a las primeras componentes principales. Aunque PCA y Random Forest responden a criterios distintos (varianza explicada frente a capacidad predictiva), la coincidencia observada refuerza la validez del modelo y su interpretación desde el punto de vista enológico.

Hito 4: Despliegue y operacionalización del sistema

Introducción

El cuarto hito del proyecto tiene como objetivo transformar el trabajo experimental realizado en los hitos anteriores en un sistema desplegable, reproducible y orientado a su uso operativo. Para ello, se ha diseñado una solución basada en contenedores que separa las fases de entrenamiento e inferencia.

En este hito se ha estructurado el proyecto en componentes independientes y configurables. Esta decisión permite encapsular dependencias, reducir problemas de portabilidad y acercar el sistema a un escenario real de despliegue.

Además del propio despliegue, este capítulo pone el foco en las decisiones de diseño adoptadas para dividir el sistema en módulos Python y contenedores diferenciados. Se justifica por qué se ha optado por esta estructura y se analizan las implicaciones técnicas que ha tenido sobre el desarrollo, la ejecución y la mantenibilidad del proyecto.

Arquitectura general del despliegue

La arquitectura implementada se compone de dos servicios principales: un contenedor de entrenamiento y un contenedor de inferencia. El primero se encarga de entrenar el modelo a partir del conjunto de datos del proyecto y almacenar el artefacto resultante; el segundo carga dicho modelo y expone una API web para realizar predicciones desde el exterior.

A nivel de repositorio, la lógica de entrenamiento e inferencia se ha desacoplado en los archivos `src/train.py` y `src/infer.py`. Además, la construcción de cada servicio se ha separado en los archivos `Dockerfile.train` y `Dockerfile.infer`, mientras que la orquestación conjunta se realiza mediante `docker-compose.yaml`.

Esta separación responde a una buena práctica habitual en sistemas de aprendizaje automático desplegados, ya que el entrenamiento y la inferencia presentan requisitos distintos. El entrenamiento suele ser una tarea puntual y más costosa computacionalmente, mientras que la inferencia debe ofrecer disponibilidad continua y tiempos de respuesta bajos.

Decisiones de diseño y modularización

Uno de los objetivos principales de este hito ha sido transformar el trabajo experimental desarrollado en formato notebook en una solución más cercana a un entorno de explotación real. Para ello, se decidió dividir la lógica del sistema en módulos Python y servicios contenerizados con responsabilidades bien diferenciadas, en lugar de mantener una implementación monolítica centrada en un único script o notebook.

La primera decisión de diseño consistió en separar las fases de entrenamiento e inferencia. Esta elección se justificó porque ambas tareas tienen naturalezas distintas: el entrenamiento es un proceso puntual, intensivo y orientado a generar un artefacto de modelo, mientras que la inferencia requiere disponibilidad continua, tiempos de respuesta bajos y capacidad para atender peticiones externas. Mantener ambas fases desacopladas facilita la mantenibilidad del sistema y evita rehacer el entrenamiento cada vez que se despliega o reinicia la API.

A nivel de código, esta decisión se materializó en la separación entre `src/train.py` y `src/infer.py`. El archivo `train.py` concentra la lógica de carga de datos, entrenamiento y serialización del modelo, mientras que `infer.py` se responsabiliza exclusivamente de cargar el artefacto entrenado y exponer los endpoints de servicio. Esta modularización reduce el acoplamiento entre componentes, mejora la claridad del repositorio y permite modificar la estrategia de entrenamiento sin alterar la capa de servicio de inferencia.

La segunda decisión relevante fue emplear dos contenedores diferentes en lugar de una imagen única para todo el sistema. La razón principal es que el contenedor de entrenamiento tiene un comportamiento transitorio, ya que se ejecuta para generar el modelo y puede finalizar una vez completada esa tarea, mientras que el contenedor de inferencia debe permanecer activo ofreciendo el servicio HTTP. Además, esta separación permite adaptar mejor cada imagen a su propósito y sienta las bases para futuros escenarios de escalado o sustitución independiente de componentes.

Como mecanismo de comunicación entre ambos servicios se optó por un volumen compartido para almacenar el modelo serializado. Esta decisión simplifica el flujo de despliegue y evita introducir, en esta fase académica del proyecto, una infraestructura adicional de almacenamiento de artefactos. La implicación directa de esta elección es que el sistema resulta sencillo de reproducir en local, aunque en un entorno productivo más avanzado podría sustituirse por un registro de modelos o un almacenamiento remoto más robusto.

Otra decisión de diseño fue cargar el modelo al inicio del contenedor de inferencia, en lugar de hacerlo en cada petición. Esta estrategia reduce la latencia de respuesta y evita operaciones repetitivas de acceso a disco, a costa de requerir que el artefacto del modelo exista previamente y que cualquier cambio en el mismo implique reiniciar el servicio o desplegar una nueva versión del contenedor.

Durante las pruebas de despliegue se observó que el arranque simultáneo de los contenedores desde un volumen vacío podía provocar que el servicio de inferencia intentase cargar el modelo antes de que el servicio de entrenamiento hubiese terminado de generarlo. Como consecuencia, el contenedor de inferencia fallaba en el arranque si el artefacto `wine_model.pkl` todavía no existía. Esta situación pone de manifiesto una condición de carrera derivada de la separación entre entrenamiento e inferencia. Por ello, el flujo operativo finalmente adoptado consistió en ejecutar primero `docker compose run -rm training` para generar el modelo y, posteriormente, `docker compose up inference` para levantar la API de inferencia sobre un artefacto ya disponible.

Finalmente, la API de inferencia se diseñó con un conjunto reducido de endpoints: `/health`, `/predict` y `/metadata`. Se optó por esta estructura porque cubre las necesidades esenciales del despliegue exigido en el hito: verificar el estado del servicio, consumir predicciones desde el exterior y exponer información básica del modelo cargado. La ventaja de esta decisión es la simplicidad y claridad del servicio.

Contenerización del sistema

Para el despliegue del sistema se ha optado por Docker como tecnología principal de contenerización. Esta decisión permite empaquetar el código, las dependencias y la configuración de ejecución en imágenes reproducibles, evitando diferencias entre entornos de desarrollo y ejecución.

El contenedor de entrenamiento se ha diseñado para ejecutar el proceso de aprendizaje del modelo y guardar el artefacto generado en una ruta compartida. De esta forma, el modelo entrenado puede ser reutilizado posteriormente por el servicio de inferencia sin necesidad de repetir el entrenamiento en cada arranque.

Por su parte, el contenedor de inferencia se ha diseñado con el objetivo de cargar el modelo previamente entrenado y mantener activo un servicio accesible desde el exterior. Esta aproximación reduce el acoplamiento entre fases y permite reiniciar o escalar la capa de inferencia sin afectar al proceso de entrenamiento.

Orquestación con Docker Compose

La ejecución conjunta de los servicios se ha gestionado mediante Docker Compose. Esta herramienta simplifica la definición de varios contenedores coordinados dentro de una misma aplicación, permitiendo especificar servicios, volúmenes, puertos y dependencias de forma declarativa.

En el archivo `docker-compose.yaml` se define la relación entre los servicios de entrenamiento e inferencia, así como el uso de almacenamiento compartido para persistir el modelo generado. Gracias a

esta configuración, el flujo de trabajo queda automatizado y puede reproducirse mediante comandos simples desde terminal.

Los comandos principales utilizados durante el desarrollo y prueba del sistema han sido los siguientes:

```
docker compose up --build
docker compose up --build inference
docker compose run --rm training
docker compose down
```

Este enfoque facilita la repetibilidad del despliegue y reduce la complejidad operativa del sistema. Además, deja preparada la base para futuras extensiones, como la incorporación de herramientas de monitorización, registro de experimentos o despliegues más avanzados sobre orquestadores.

Servicio de inferencia

Uno de los elementos centrales del Hito 4 es la exposición del modelo mediante una API accesible desde el exterior. En esta implementación, el contenedor de inferencia ofrece una interfaz HTTP para consultar el estado del servicio, obtener metadatos del modelo y realizar predicciones a partir de nuevos datos de entrada.

La API se ha concebido para desacoplar el modelo del cliente consumidor. De este modo, cualquier aplicación externa capaz de enviar peticiones HTTP puede reutilizar el sistema sin necesidad de conocer detalles internos del entrenamiento ni del formato de almacenamiento del modelo.

El endpoint de predicción valida automáticamente el formato de entrada mediante un esquema definido con Pydantic en FastAPI. De este modo, el servicio garantiza que las trece variables físicoquímicas requeridas se reciban con el nombre y tipo esperados antes de ejecutar la predicción.

De forma general, el servicio dispone de tres endpoints principales:

- GET /health: permite comprobar si el servicio está activo.
- GET /metadata: devuelve información descriptiva del modelo cargado.
- POST /predict: recibe un conjunto de variables de entrada y devuelve la predicción correspondiente.

Para verificar el funcionamiento del sistema se realizaron pruebas locales desde terminal sobre el puerto expuesto por el contenedor de inferencia. Un ejemplo representativo de invocación es el siguiente:

```
curl -X POST http://localhost:8000/predict \  
-H "Content-Type: application/json" \  
-d '{"Alcohol":13.2,"Malic_Acid":1.78,"Ash":2.14,  
"Alcalinity_of_Ash":11.2,"Magnesium":100,"Total_Phenols":2.65,  
"Flavanoids":2.76,"Nonflavanoid_Phenols":0.26,  
"Proanthocyanins":1.28,"Color_Intensity":4.38,  
"Hue":1.05,"OD280_OD315":3.40,"Proline":1050}'
```

Estas pruebas permitieron validar tanto la correcta carga del modelo como la accesibilidad externa del servicio. En consecuencia, el sistema queda preparado para integrarse con clientes ligeros e interfaces web.

Análisis de necesidades del sistema

Desde el punto de vista de recursos, la solución desarrollada no requiere aceleración mediante GPU. El modelo empleado pertenece al ámbito del aprendizaje automático clásico y puede ejecutarse de forma adecuada sobre CPU en un entorno local.

En cuanto al consumo de memoria y almacenamiento, los requisitos del sistema son moderados. El conjunto de datos utilizado tiene un tamaño reducido y el artefacto resultante del entrenamiento ocupa poco espacio, por lo que no ha sido necesario recurrir a soluciones distribuidas ni a almacenamiento especializado para esta versión del proyecto.

Respecto al patrón de uso, el sistema se ajusta a un escenario de inferencia online bajo demanda. Esto significa que el servicio permanece disponible para responder a peticiones puntuales, sin necesidad de ejecutar lotes masivos ni flujos de streaming continuo. En un contexto de producción, esta arquitectura podría ampliarse con mecanismos de autenticación, monitorización y escalado horizontal.

Bonus: integración con MLflow

Como extensión opcional del hito, se ha implementado un entorno independiente de seguimiento de experimentos mediante MLflow en la subcarpeta `bonus_mlflow`. Esta parte del trabajo se ha mantenido desacoplada del despliegue principal de entrenamiento e inferencia, pero lo complementa con capacidades de trazabilidad, registro de ejecuciones y almacenamiento de artefactos.

En este bonus se desplegó un servidor local de MLflow mediante Docker Compose y se ejecutó un script específico de entrenamiento, `train_mlflow.py`, capaz de registrar automáticamente información del experimento. Entre los elementos almacenados se incluyen el nombre del experimento, la

ejecución realizada, los parámetros del modelo, las métricas obtenidas y el modelo generado como artefacto.

La interfaz web de MLflow quedó accesible localmente en el puerto 5000, lo que permitió consultar visualmente la ejecución registrada. Esta integración añade una capa de MLOps al proyecto, ya que facilita la comparación de experimentos, la trazabilidad del entrenamiento y la gestión de artefactos de modelo.

La evidencia visual del experimento, de la ejecución registrada y del modelo almacenado puede consultarse en las Figuras 5, 6 y 7.

Evidencias del bonus MLflow

Las Figuras 5, 6 y 7 muestran, respectivamente, la vista general del experimento, el detalle de la ejecución registrada y la información del modelo almacenado en MLflow. Estas capturas sirven como evidencia visual de que el servidor de seguimiento quedó correctamente desplegado y de que el experimento fue registrado de forma satisfactoria.

En la Figura 5 puede observarse el experimento DISIA Bonus MLflow con la ejecución wine_random_forest_bonus registrada correctamente. Esta vista resulta útil para mostrar de forma global la relación entre experimento, ejecución y modelo asociado.

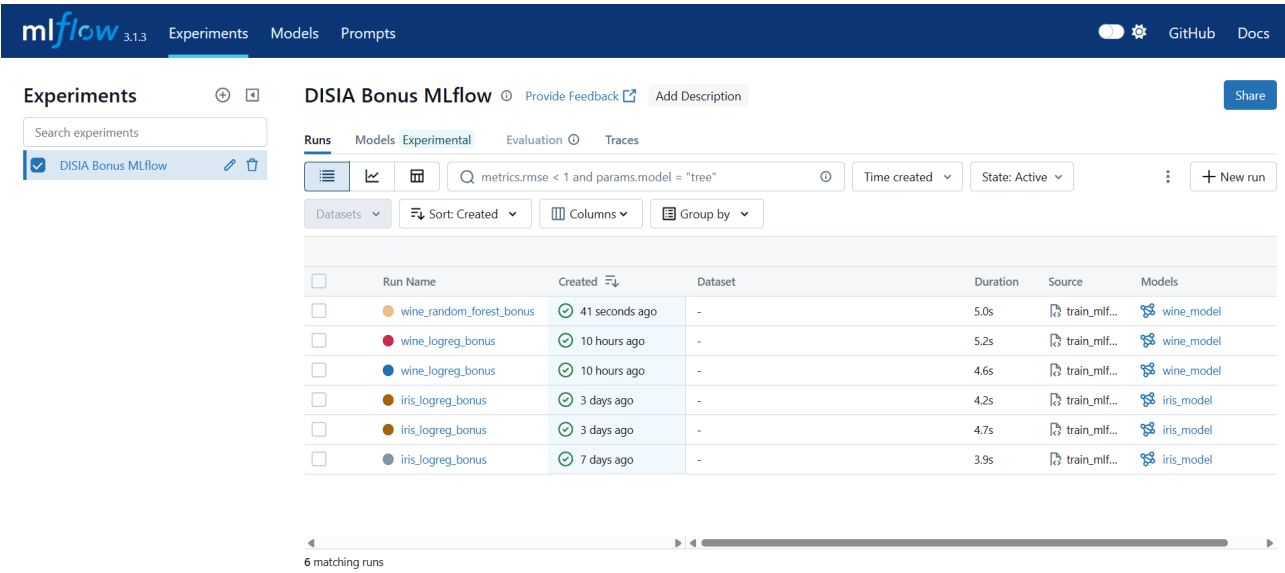


Figura 5: Vista general del experimento registrado en MLflow.

La Figura 6 presenta el detalle de la ejecución, incluyendo métricas, parámetros y metadatos de la run. Esta captura permite comprobar que el entrenamiento no solo se ejecutó correctamente, sino que además quedó documentado con información reproducible sobre la configuración utilizada.

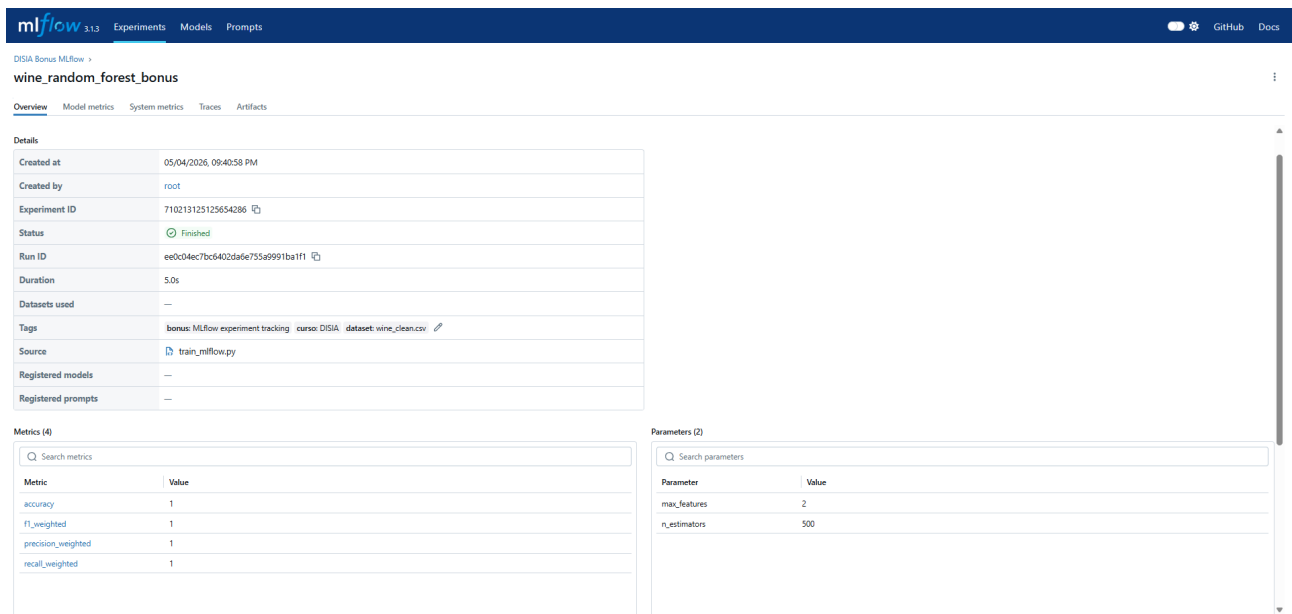


Figura 6: Detalle de la ejecución registrada, incluyendo métricas y parámetros.

Finalmente, la Figura 7 muestra el modelo almacenado en MLflow y vinculado a la ejecución de origen. Esto demuestra que el bonus no se limitó al registro de métricas, sino que también incluyó el almacenamiento del artefacto del modelo junto con su trazabilidad experimental.

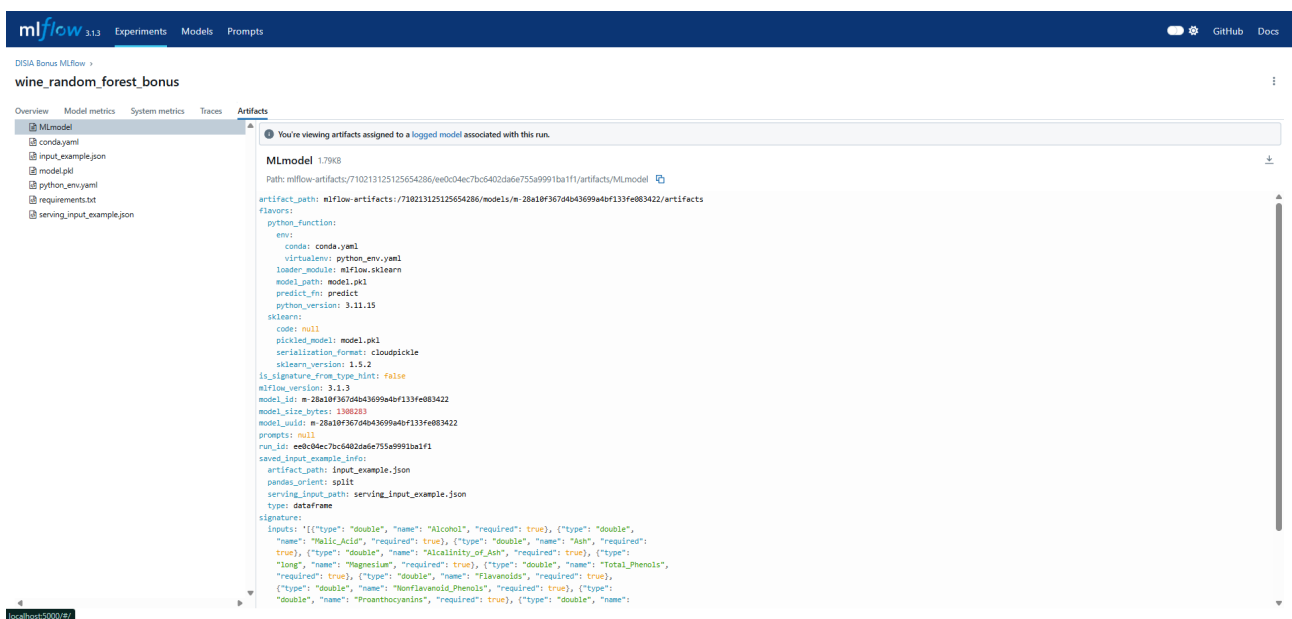


Figura 7: Modelo almacenado en MLflow y vinculado a la ejecución de origen.

Valoración final del hito

El Hito 4 ha permitido transformar el trabajo previo de modelado en un sistema desplegable y reutilizable. La separación entre entrenamiento e inferencia, junto con la contenerización del proyecto,

mejora la organización del código y aproxima la solución a un entorno real de explotación.

Adicionalmente, la incorporación de MLflow como bonus aporta una primera capa de seguimiento experimental propia de flujos MLOps. Aunque se trata de una implementación local y académica, el resultado demuestra la viabilidad de evolucionar el proyecto hacia arquitecturas más completas de monitorización, versionado y despliegue. En conjunto, las decisiones de modularización adoptadas han incrementado ligeramente la complejidad estructural del proyecto, pero a cambio han mejorado su mantenibilidad, su claridad arquitectónica y su proximidad a un escenario real de operación.

Hito 5: Monitorización, feedback loop y shadow testing

Introducción

El quinto hito del proyecto tiene como objetivo ampliar el sistema desplegado en el Hito 4 con capacidades de observabilidad, monitorización y mejora continua propias de un flujo MLOps. Para ello, se ha diseñado una solución orientada a supervisar el comportamiento operativo del servicio, detectar posibles signos de degradación del modelo en producción y cerrar el ciclo mediante un mecanismo de realimentación y reentrenamiento.

A diferencia de los hitos anteriores, en esta fase el foco no se sitúa únicamente en desplegar un modelo funcional, sino en dotar al sistema de herramientas que permitan operar con mayor seguridad una vez puesto en servicio. En particular, se ha abordado el problema de cómo monitorizar un clasificador cuando la etiqueta real de cada muestra no está disponible en el instante de la predicción, situación habitual en escenarios reales de explotación.

En el contexto de VinOps, la clase real del cultivar no puede conocerse en tiempo real, ya que su validación depende de la revisión posterior del enólogo. Esta restricción impide calcular de manera inmediata métricas clásicas de rendimiento como el F1-score, la precisión o el AUC. Por este motivo, en este hito se ha optado por complementar la inferencia con métricas proxy, detección de drift, alertas automáticas y un mecanismo de feedback que permita incorporar correcciones humanas al ciclo de vida del modelo.

Además del desarrollo técnico, este capítulo pone el foco en las decisiones de diseño adoptadas para mantener una solución ligera, reproducible y coherente con el carácter académico del proyecto. Se justifica por qué se ha optado por determinadas herramientas y estrategias, y se analizan las implicaciones que dichas decisiones han tenido sobre la mantenibilidad, la interpretabilidad y la operatividad del sistema.

Arquitectura general del sistema de observabilidad

La arquitectura implementada extiende el servicio de inferencia desarrollado en el hito anterior mediante una capa adicional de instrumentación, trazabilidad y supervisión. El sistema se articula alrededor de una API desarrollada con FastAPI, accesible en el puerto 8001, que expone tanto el endpoint

de predicción como varios endpoints auxiliares orientados a la monitorización, el feedback y el reentrenamiento.

A nivel funcional, cuando un cliente envía una muestra a `/predict`, el sistema realiza la inferencia, calcula métricas proxy de calidad de la predicción, evalúa si la entrada presenta rasgos anómalos respecto a la distribución de entrenamiento, registra la ejecución en un log estructurado y actualiza las métricas expuestas para Prometheus. Sobre esta base, Grafana permite visualizar el comportamiento del servicio, mientras que un sistema de alertas por Telegram notifica situaciones relevantes al equipo.

La solución se ha desacoplado del despliegue del Hito 4 mediante una carpeta específica de observabilidad. Esta decisión permite mantener separado el sistema base de inferencia del entorno ampliado de monitorización, facilitando la revisión académica y evitando introducir complejidad innecesaria sobre el despliegue mínimo ya validado.

De forma general, los endpoints principales del servicio son los siguientes:

Tabla 20: Endpoints principales del sistema monitorizado

Endpoint	Método	Función principal
<code>/health</code>	GET	Verificar el estado del servicio, exponer metadatos básicos del modelo y resumir información operativa.
<code>/predict</code>	POST	Realizar la predicción, calcular métricas proxy, detectar anomalías y registrar la inferencia.
<code>/metrics</code>	GET	Exponer las métricas en formato Prometheus para su recolección externa.
<code>/monitor</code>	GET	Consultar un resumen agregado de actividad, anomalías y calidad de predicción.
<code>/feedback</code>	POST	Registrar correcciones proporcionadas por el enólogo para alimentar el ciclo de mejora.
<code>/retrain</code>	POST	Lanzar un reentrenamiento a partir del dataset original y del feedback acumulado.
<code>/reload</code>	POST	Recargar en memoria el modelo actualizado sin reiniciar el contenedor.

Como ampliación opcional, se ha implementado también un entorno de *shadow testing* en un puerto independiente, orientado a comparar un modelo en producción con un modelo candidato sin afectar a la respuesta devuelta al cliente. Esta extensión se comenta más adelante como bonus del hito.

Las Figuras 8 y 9 ilustran tanto el despliegue efectivo del entorno como la disponibilidad del servicio principal.

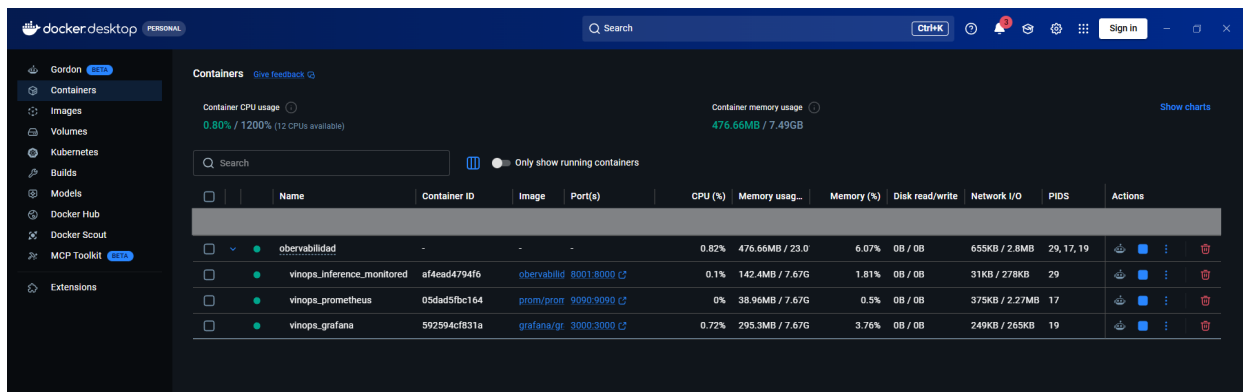


Figura 8: Entorno de observabilidad desplegado en Docker con la API monitorizada y los servicios auxiliares activos.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> Invoke-RestMethod -Uri "http://localhost:8001/health" -Method Get

status      : ok
service     : vinops-inference-monitored
version     : 5.5.0
model       : @({model_path=/app/models/wine_model.pkl; model_type=Pipeline; loaded_at=2026-05-13T08:55:02.618135})
monitor_summary : @({predictions_total=0; anomalies_total=0; low_confidence_total=0; avg_confidence=0.0; class_distribution=; uptime_seconds=540.9})
drift_summary  : @({status=NOT_EVALUATED_YET; window_size=0})
feedback_summary : @({count=10; retrain_recommended=True; retrain_threshold=10; errors_detected=4; error_rate=0.4; last_feedback=2026-05-12T23:20:09.776237; class_distribution=})
cpu_percent   : 0.0
memory_bytes  : 195997696.0
alerts_enabled : True
```

Figura 9: Respuesta del endpoint /health del servicio principal en el puerto 8001.

Instrumentación y trazabilidad del servicio

Uno de los objetivos centrales del hito ha sido convertir la API de inferencia en un servicio observable. Para ello, se han instrumentado métricas operativas habituales en sistemas software, tales como número de peticiones, latencia, uso de CPU y consumo de memoria. Esta instrumentación permite supervisar no solo el comportamiento del modelo, sino también el estado general del servicio que lo ejecuta.

La elección de Prometheus como solución de monitorización responde a varias razones. En primer lugar, se trata de una herramienta utilizada en entornos cloud-native y con integración nativa con Grafana. En segundo lugar, el cliente de Python resulta ligero y sencillo de integrar en FastAPI. Finalmente, el formato textual de las métricas facilita tanto la depuración como la validación manual durante una demostración académica.

Las principales métricas operativas instrumentadas han sido las presentados en la Tabla 21.

Junto a las métricas operativas, se ha considerado necesario instrumentar también métricas proxy del modelo. La razón es que, en el dominio del proyecto, la etiqueta real no está disponible en el instante de la predicción, por lo que no es posible calcular online métricas de rendimiento supervisadas.

Tabla 21: Métricas operativas instrumentadas

Métrica	Tipo	Descripción
vinops_requests_total	Counter	Cuenta peticiones HTTP por método, endpoint y código de estado.
vinops_request_latency_seconds	Histogram	Mide la latencia de las peticiones, especialmente en inferencia.
vinops_cpu_usage_percent	Gauge	Registra el uso de CPU del proceso que ejecuta el servicio.
vinops_memory_usage_bytes	Gauge	Registra el consumo de memoria residente del proceso.

En consecuencia, se ha optado por monitorizar señales indirectas que permitan detectar comportamientos sospechosos antes de disponer del *ground truth*.

Entre estas métricas proxy, presentadas en la Tabla 22, destacan la confianza de la predicción, la entropía de la distribución de probabilidades, el conteo de predicciones por clase, la tasa de muestras anómalas y el número de predicciones con confianza baja. La confianza y la entropía resultan especialmente útiles porque permiten detectar cuándo el modelo se encuentra en una zona de mayor incertidumbre, algo relevante en este problema debido al solapamiento ya observado entre algunas clases en el Hito 3.

Tabla 22: Métricas proxy del modelo

Métrica	Tipo	Interpretación
vinops_prediction_confidence	Histogram	Mide la probabilidad máxima asignada por el modelo a la clase predicha.
vinops_prediction_entropy	Histogram	Evalúa el grado de dispersión de la distribución de probabilidades entre clases.
vinops_predictions_by_class_total	Counter	Permite analizar cambios en la distribución de clases predichas.
vinops_anomaly_flags_total	Counter	Cuenta las muestras detectadas como anómalas respecto a la referencia.
vinops_low_confidence_total	Counter	Cuenta predicciones cuya confianza cae por debajo del umbral establecido.

Otra decisión de diseño relevante ha sido reutilizar en inferencia la lógica de detección de anomalías introducida en el Hito 2. Para cada muestra entrante se calcula el z-score de cada variable respecto a la media y desviación estándar del conjunto de entrenamiento, siguiendo la expresión:

$$z_j = \frac{x_j - \bar{x}_j}{\sigma_j}, \quad j = 1, \dots, 13$$

Si alguna variable supera en valor absoluto el umbral de tres desviaciones estándar, la muestra se marca como anómala. Esta estrategia aporta coherencia entre el análisis exploratorio previo y la supervisión operativa del sistema, además de ofrecer una forma simple y explicable de identificar entradas poco representativas.

La trazabilidad de las predicciones se completa mediante un sistema de *logging* estructurado en formato JSONL. Cada inferencia genera una línea independiente que incluye el instante temporal, las variables de entrada, la clase predicha, las probabilidades por clase, la confianza, la entropía, el resultado del detector de anomalías y la latencia de la respuesta. Se ha optado por JSONL porque permite escritura incremental eficiente sin necesidad de reescribir estructuras completas, y facilita el análisis posterior con herramientas de línea de comandos o librerías como pandas.

Las Figuras 10 y 11 ilustran dos situaciones representativas del servicio de inferencia: una predicción ordinaria con comportamiento esperado y una muestra con rasgos anómalos detectados por el monitor.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> $body = @{
    >> Alcohol = 13.2
    >> MalicAcid = 1.78
    >> Ash = 2.14
    >> AlcalinityAsh = 11.2
    >> Magnesium = 180
    >> TotalPhenols = 2.65
    >> Flavanoids = 2.76
    >> NonFlavanoidPhenols = 0.26
    >> Proanthocyanins = 1.28
    >> ColorIntensity = 4.38
    >> Hue = 1.05
    >> OD280_OD315 = 3.40
    >> Proline = 1050
    >> } | ConvertTo-Json
* >>> Invoke-RestMethod -Uri "http://localhost:8081/predict" -Method Post -ContentType "application/json" -Body $body

prediction          : 2
class_label         : cultivar_2
probabilities       : @{class_1=0.3569; class_2=0.6431; class_3=0.0}
confidence          : 0.6431
entropy             : 0.9401
anomaly             : @{is_anomaly=False; anomalous_variables=System.Object[]; z_scores=-; threshold=3.0}
is_low_confidence   : True
model_version       : 2026-05-13T08:55:02.618135
latency_ms          : 47,1
```

Figura 10: Ejemplo de predicción normal realizada por el endpoint /predict.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> $body_anom = @{
    >> Alcohol = 18.5
    >> MalicAcid = 7.5
    >> Ash = 4.5
    >> AlcalinityAsh = 40
    >> Magnesium = 180
    >> TotalPhenols = 0.2
    >> Flavanoids = 0.1
    >> NonFlavanoidPhenols = 1.2
    >> Proanthocyanins = 4.5
    >> ColorIntensity = 18
    >> Hue = 0.2
    >> OD280_OD315 = 0.5
    >> Proline = 2000
    >> } | ConvertTo-Json
* >>> Invoke-RestMethod -Uri "http://localhost:8081/predict" -Method Post -ContentType "application/json" -Body $body_anom
Invoke-RestMethod : {"detail":[{"type":"less_than_equal","loc":["body","Alcohol"],"msg":"Input should be less than or equal to
16","input":18.5,"ctx":{"le":16.0}},{"type":"less_than_equal","loc":["body","MalicAcid"],"msg":"Input should be less than or equal to
6","input":7.5,"ctx":{"le":6.0}},{"type":"less_than_equal","loc":["body","Ash"],"msg":"Input should be less than or equal to
4","input":4.5,"ctx":{"le":4.0}},{"type":"less_than_equal","loc":["body","AlcalinityAsh"],"msg":"Input should be less than or equal to
35","input":40,"ctx":{"le":35.0}},{"type":"greater_than_equal","loc":["body","TotalPhenols"],"msg":"Input should be greater than or equal to
0.5","input":0.2,"ctx":{"ge":0.5}},{"type":"less_than_equal","loc":["body","NonFlavanoidPhenols"],"msg":"Input should be less than or equal to
1","input":1.2,"ctx":{"le":1.0}},{"type":"less_than_equal","loc":["body","Proanthocyanins"],"msg":"Input should be less than or equal to
4","input":4.5,"ctx":{"le":4.0}},{"type":"less_than_equal","loc":["body","ColorIntensity"],"msg":"Input should be less than or equal to
14","input":18,"ctx":{"le":14.0}},{"type":"greater_than_equal","loc":["body","Hue"],"msg":"Input should be greater than or equal to
0.3","input":0.2,"ctx":{"ge":0.3}},{"type":"greater_than_equal","loc":["body","OD280_OD315"],"msg":"Input should be greater than or equal to
1","input":0.5,"ctx":{"ge":1.0}},{"type":"less_than_equal","loc":["body","Proline"],"msg":"Input should be less than or equal to 1700","input":2000,"ctx":{"le":1700.0}}]}
En línea: 17 Carácter: 1
+ Invoke-RestMethod -Uri "http://localhost:8081/predict" -Method Post - ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

Figura 11: Ejemplo de predicción anómala con indicadores activados en la respuesta del servicio.

Detección de drift y sistema de alertas

Más allá de supervisar cada petición individual, el sistema necesita detectar cambios sostenidos en la distribución de los datos de entrada. En este contexto se ha optado por centrar el análisis en el

data drift o deriva de covariables, es decir, en el cambio de la distribución $P(X)$ entre la fase de entrenamiento y la fase de operación. Esta elección resulta especialmente adecuada para VinOps porque puede evaluarse sin necesidad de disponer de la etiqueta real de cada muestra.

La Tabla 23 presenta tres técnicas complementarias utilizadas en la implementación. En primer lugar, la prueba de Kolmogorov-Smirnov permite comparar distribuciones continuas de forma no paramétrica para cada variable individual. En segundo lugar, el *Population Stability Index* proporciona una señal interpretable sobre la magnitud del desplazamiento entre distribuciones. En tercer lugar, la distancia de Wasserstein estandarizada ofrece una visión agregada del desplazamiento conjunto de la ventana observada.

Tabla 23: Técnicas de detección de drift implementadas

Técnica	Tipo	Umbral	Finalidad principal
Kolmogorov-Smirnov	Univariada	$p < 0,05$	Detectar cambios estadísticamente significativos en variables individuales.
PSI	Univariada	$> 0,10$ monitor, $> 0,25$ alerta	Medir estabilidad poblacional con una regla fácilmente interpretable.
Wasserstein	Multivariada	$> 1,0$ monitor, $> 2,0$ alerta	Capturar desplazamientos globales que podrían no apreciarse variable a variable.

Con el fin de equilibrar sensibilidad y coste computacional, se ha configurado una ventana deslizando de veinte muestras y una evaluación periódica cada diez predicciones. Esta decisión se justifica por el reducido tamaño del dataset original, ya que ventanas demasiado grandes introducirían una detección excesivamente lenta y ventanas demasiado pequeñas generarían señales muy ruidosas. La información de drift se resume mediante una regla jerárquica que diferencia entre estado normal, estado de monitorización y deriva detectada.

De forma simplificada, el sistema considera que existe deriva cuando se acumulan evidencias suficientes en varias variables o cuando el desplazamiento global medido por Wasserstein supera el umbral crítico. Esta combinación reduce el riesgo de falsos positivos debidos a fluctuaciones puntuales en una sola característica y aporta mayor robustez a la decisión global del monitor.

Sobre esta base se ha implementado un sistema de alertas por Telegram. Se eligió este canal por su simplicidad a la hora de integrarlo, por no requerir infraestructura propia y por resultar especialmente adecuado para una demostración en vivo, ya que permite visualizar las notificaciones de forma inmediata desde un chat. Además, el uso de variables de entorno para el token y el `chat_id` evita exponer credenciales en el código fuente.

Con el objetivo de evitar saturación informativa, se ha incorporado un mecanismo de *throttling* de sesenta segundos por tipo de alerta. Esta decisión reduce el riesgo de spam cuando una condición anómala persiste durante varios ciclos de monitorización, manteniendo al mismo tiempo una ca-

dencia suficientemente frecuente para no perder capacidad de reacción.

Las alertas implementadas cubren dos familias de necesidades. Por un lado, se emiten alertas de modelo cuando se detecta drift, una tasa alta de anomalías o una caída sostenida de confianza media. Por otro lado, se generan alertas operativas cuando el servicio supera ciertos umbrales de CPU o memoria. A ello se añaden endpoints de simulación, como `/simulate/anomaly` y `/simulate/drift`, que permiten provocar escenarios controlados y verificar experimentalmente el funcionamiento del detector y del canal de notificación.

Con el fin de validar experimentalmente el sistema de monitorización, se ejecutaron simulaciones controladas que permitieron observar tanto la activación del canal de alertas como la actualización del estado de deriva del sistema. Las alertas enviadas por el bot de Telegram puede verse en las Figuras 12 y 13, además del estado del estado del drift en la Figura 14.

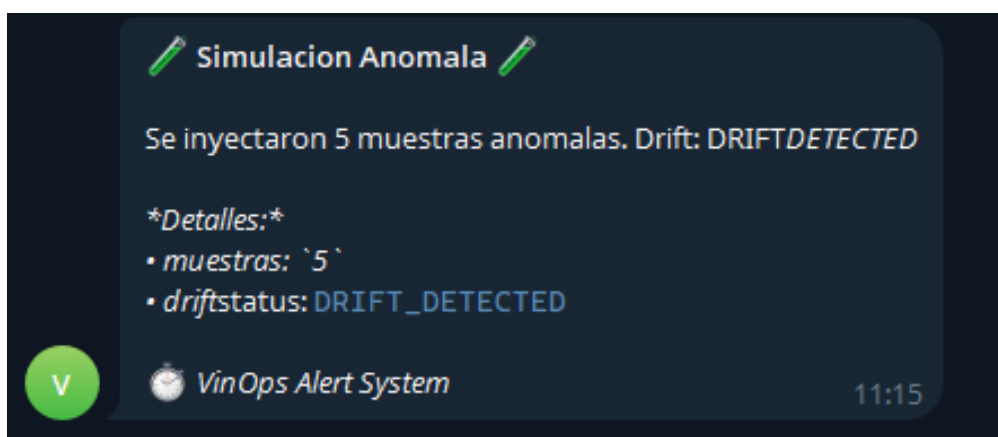


Figura 12: Alerta enviada por Telegram tras detectar un comportamiento anómalo en el sistema monitorizado.

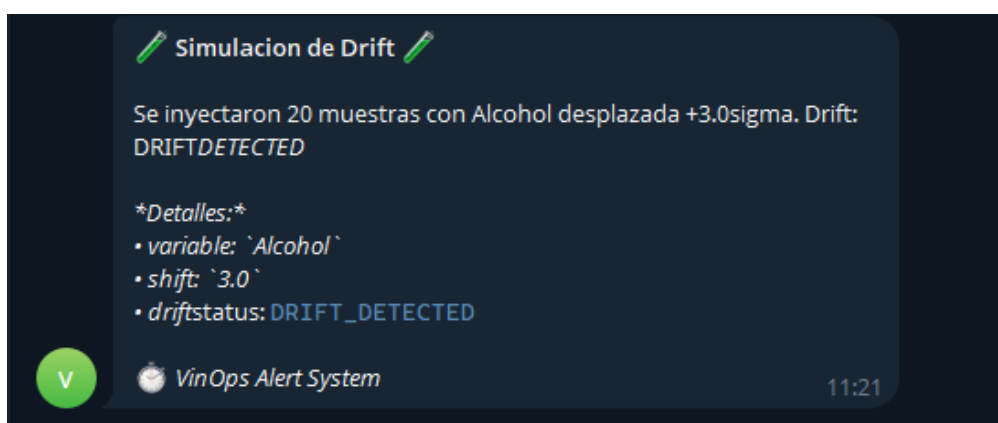


Figura 13: Notificación de deriva enviada al canal de Telegram del proyecto.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> Invoke-RestMethod -Uri "http://localhost:8001/dri
ft" -Method Get

timestamp : 2026-05-13T09:21:43.246188
window_size : 20
status : DRIFT DETECTED
recommendation : RETRAIN
ks_test : @(alert_threshold=0.05; variables_alerted=System.Object[]; num_alerted=7; details=)
psi : @(alert_threshold=0.25; monitor_threshold=0.1; variables_alerted=System.Object[]; num_alerted=13; details=)
wasserstein : @(distance=0.6506; alert_threshold=2.0; monitor_threshold=1.0)
```

Figura 14: Estado de drift del sistema tras la simulación, mostrando activación del monitor y recomendación de actuación.

Visualización con Prometheus y Grafana

La solución de visualización se ha articulado en tres capas. La primera capa corresponde a la propia API, que emite las métricas instrumentadas a través del endpoint `/metrics`. La segunda capa la constituye Prometheus, que scrapea periódicamente dichas métricas y las almacena como series temporales. Por último, Grafana consume esos datos y los presenta en un dashboard orientado a la monitorización del servicio.

El dashboard diseñado para el proyecto integra paneles dedicados a CPU, memoria, peticiones por segundo, latencia, confianza media del modelo, distribución de clases predichas, tasa de anomalías y entropía de predicción. Esta selección responde al objetivo de combinar indicadores operativos clásicos con señales específicas del comportamiento del clasificador, de modo que el operador disponga de una visión más completa del estado del sistema.

Una observación relevante durante las pruebas es que el uso de CPU aparece próximo a cero en condiciones normales. Este comportamiento no se debe a un error de instrumentación, sino a que el modelo empleado es muy ligero y la inferencia sobre un dataset pequeño apenas consume recursos en cada petición individual. En consecuencia, el valor del gauge solo crecería de forma apreciable bajo una carga artificialmente elevada.

La evidencia experimental de esta capa puede verse en las Figuras 15 y 16.

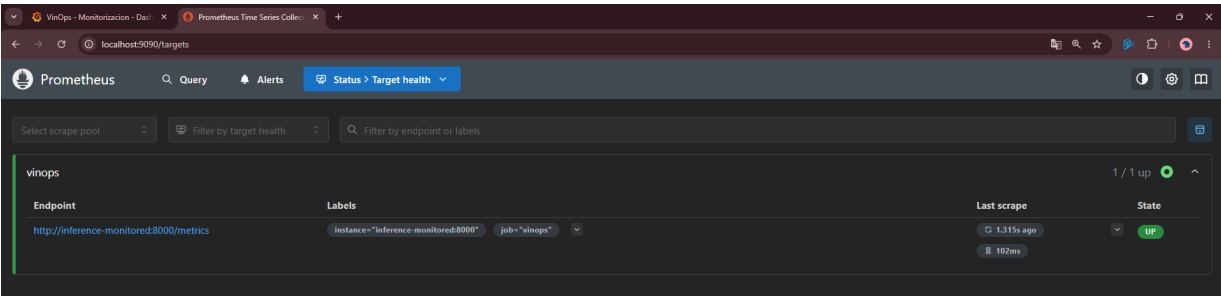


Figura 15: Vista de Prometheus con el target del servicio en estado UP.

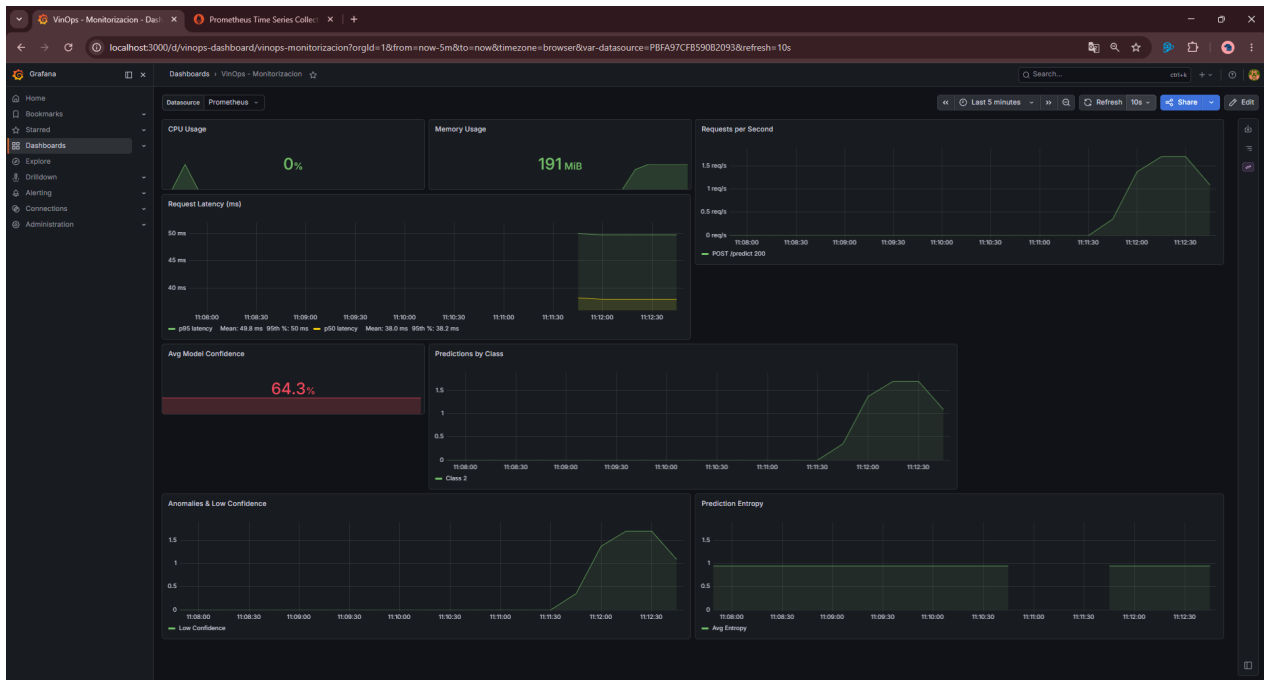


Figura 16: Dashboard de Grafana con paneles de latencia, memoria, confianza y distribución de clases.

Feedback loop y retraining semi-automático

Uno de los elementos más importantes del Hito 5 es el cierre parcial del ciclo de vida del modelo mediante un mecanismo de feedback humano. La idea central consiste en permitir que el enólogo corrija predicciones una vez dispone de la etiqueta real, almacenando dichas correcciones para su uso posterior en un reentrenamiento supervisado. De este modo, el sistema no solo detecta degradación, sino que también incorpora una vía explícita para adaptarse a nuevos datos.

El diseño del trigger de reentrenamiento constituye una decisión relevante. En lugar de activar el retraining cuando se detecta un cierto número de eventos de drift, se ha optado por utilizar como criterio la acumulación de diez feedbacks reales. La razón es que el drift indica cambio en la distribución de entrada, pero no aporta por sí mismo la etiqueta correcta de las nuevas muestras. Sin *ground truth*, un reentrenamiento supervisado carecería de base válida.

Las correcciones se almacenan en un archivo CSV en lugar de una base de datos relacional. Esta elección se justifica por la simplicidad del escenario, por el reducido volumen esperado de registros y por la facilidad con la que el archivo puede revisarse manualmente durante la evaluación académica. Aunque una base de datos ofrecería mejores garantías de escalabilidad y concurrencia, su incorporación habría añadido complejidad innecesaria para este contexto.

Otra decisión especialmente importante ha sido no reentrenar el modelo únicamente con las muestras corregidas. Dado que el dataset original contiene 178 observaciones, usar solo diez ejemplos de feedback conduciría a un riesgo elevado de sobreajuste y *catastrophic forgetting*. Por este motivo, el

sistema genera un dataset combinado formado por los datos originales más el feedback duplicado, otorgando así mayor peso a las correcciones recientes sin destruir el conocimiento aprendido previamente.

$$\text{Dataset final} = \text{Original} + 2 \times \text{Feedback}$$

En el caso evaluado, el conjunto original de 178 muestras se combina con diez correcciones duplicadas, produciendo un total de 198 observaciones para el reentrenamiento. Esta estrategia permite introducir adaptación al contexto operativo manteniendo al mismo tiempo una base suficientemente estable sobre la que apoyar la generalización del modelo. La duplicación del feedback se interpreta como una ponderación simple de las muestras recientes, no como generación de nuevas observaciones. En una versión productiva, esta ponderación debería parametrizarse y validarse empíricamente comparando distintos pesos del feedback.

Tras el reentrenamiento, el nuevo artefacto se recarga en memoria mediante el endpoint `/reload`, sin necesidad de reiniciar el contenedor. Esta decisión mejora la disponibilidad del sistema, preserva las métricas acumuladas por Prometheus y evita perder tanto la ventana de drift como el contexto operativo de la ejecución en curso.

La operativa del *feedback loop* se validó mediante el envío manual de correcciones al endpoint `/feedback`, su persistencia en CSV y el posterior lanzamiento del proceso de reentrenamiento. Las Figuras 17–21 recogen la secuencia completa de corrección, almacenamiento, reentrenamiento y recarga del modelo.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> $feedback = @{
>>   features = @(
>>     Alcohol = 13.2
>>     MalicAcid = 1.78
>>     Ash = 2.14
>>     Alcalinity\Ash = 11.2
>>     Magnesium = 100
>>     TotalPhenols = 2.65
>>     Flavanoids = 2.76
>>     NonFlavanoidPhenols = 0.26
>>     Proanthocyanins = 1.28
>>     ColorIntensity = 4.38
>>     Hue = 1.45
>>     OD280_OD315 = 3.40
>>     Proline = 1050
>>   )
>>   predicted_class = 2
>>   true_class = 1
>> } | ConvertTo-Json -Depth 3
>> Invoke-RestMethod -Uri "http://localhost:8081/Feedback" -Method Post -ContentType "application/json" -Body $feedback

status      : feedback_stored
feedback_count : 23
retrain_recommended : True
predicted    : 2
true         : 1
match        : False
```

Figura 17: Registro de feedback manual enviado al sistema para alimentar el ciclo de mejora.


```

PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> Get-Content .\feedback\feedback.csv
Alcohol,Malic_Acid,Ash,Alcalinity_of_Ash,Magnesium,Total_Phenols,Flavanoids,Nonflavanoid_Phenols,Proanthocyanins,Color_Intensity,Hue,OD280_OD315,Proline,predicted_class,true_class,timesta
mp
14.23,1.71,2.43,15.6,127.0,2.8,3.06,0.28,2.29,5.64,1.04,3.92,1065.0,1.1,2026-05-12T23:20:08.474228
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-12T23:20:08.607825
13.16,2.36,2.67,18.6,101.0,2.8,3.24,0.3,2.81,5.68,1.03,3.17,1185.0,2.2,2026-05-12T23:20:08.729534
14.37,1.95,2.5,16.8,113.0,3.85,3.49,0.24,2.18,7.8,0.86,3.45,1480.0,1.1,2026-05-12T23:20:08.854985
13.24,2.59,2.87,21.0,118.0,2.8,2.69,0.39,1.82,4.32,1.04,2.93,735.0,2.3,2026-05-12T23:20:09.026397
14.2,1.76,2.45,15.2,112.0,3.27,3.39,0.34,1.97,6.75,1.05,2.85,1450.0,1.1,2026-05-12T23:20:09.196184
14.39,1.87,2.45,14.6,96.0,2.5,2.52,0.3,1.98,5.25,1.02,3.58,1290.0,1.2,2026-05-12T23:20:09.366466
14.06,2.15,2.61,17.6,121.0,2.6,2.51,0.31,1.25,5.05,1.06,3.58,1295.0,2.2,2026-05-12T23:20:09.487188
12.85,1.6,2.52,17.7,95.0,2.48,2.37,0.26,1.46,3.93,1.09,3.63,1015.0,3.3,2026-05-12T23:20:09.613370
13.5,1.81,2.65,19.0,95.0,2.2,2.43,0.26,1.57,3.8,1.13,3.71,1010.0,3.2,2026-05-12T23:20:09.776237
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:20.984163
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:56.947370
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.013887
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.073591
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.133793
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.203684
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.284228
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.344096
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.359741
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.423663
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:25:57.483713
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:26:01.753694
13.2,1.78,2.14,11.2,100.0,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050.0,2.1,2026-05-13T09:27:10.195756

```

Figura 18: Persistencia del feedback acumulado en el archivo CSV del sistema.

```

PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> Invoke-RestMethod -Uri "http://localhost:8001/retrain" -Method Post

status      model_metadata                                     feedback_used output
-----
retrained_and_reloaded @ (model_path=app\models\wine_model.pkl; model_type=Pipeline; loaded_at=2026-05-13T09:28:03.156176) 23 'Ash', 'AlcalinityAsh', 'Magnesium', 'TotalPhenols', 'Fla...

```

Figura 19: Salida del endpoint /retrain mostrando la ejecución del reentrenamiento y sus resultados.

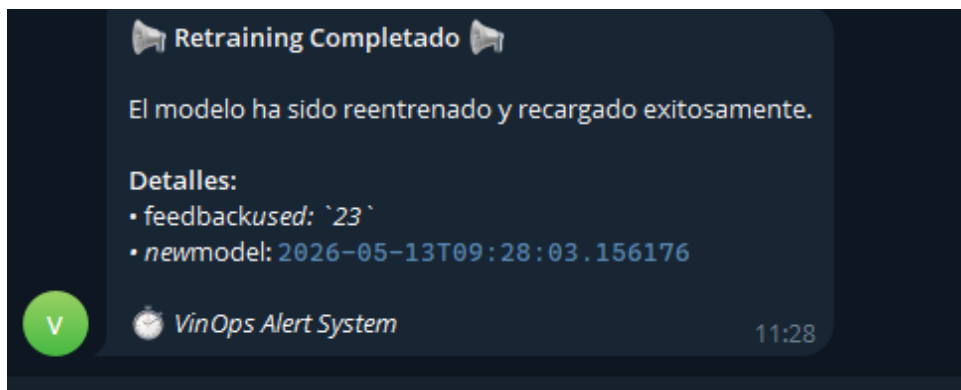


Figura 20: Notificación emitida tras la ejecución del proceso de reentrenamiento.

```

PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA\observabilidad> Invoke-RestMethod -Uri "http://localhost:8001/reload" -Method Post

status      previous_version      new_version      model_type
-----
reloaded 2026-05-13T09:28:03.156176 2026-05-13T09:29:13.318874 Pipeline

```

Figura 21: Recarga del nuevo modelo en memoria mediante /reload sin reiniciar el contenedor.

Los resultados obtenidos en el escenario descrito se resumen en la Tabla 24. Aunque se trata de una prueba académica y no de un proceso continuo real, el experimento demuestra la viabilidad del enfoque adoptado para cerrar parcialmente el ciclo de vida del modelo.

Tabla 24: Resumen del retraining con feedback acumulado

Elemento	Valor
Dataset original	178 muestras
Feedback acumulado	10 muestras
Feedback tras duplicado	20 muestras
Dataset combinado	198 muestras
Macro F1-Score	0.8757
Balanced Accuracy	0.8767

Bonus: shadow testing y comparación champion-challenger

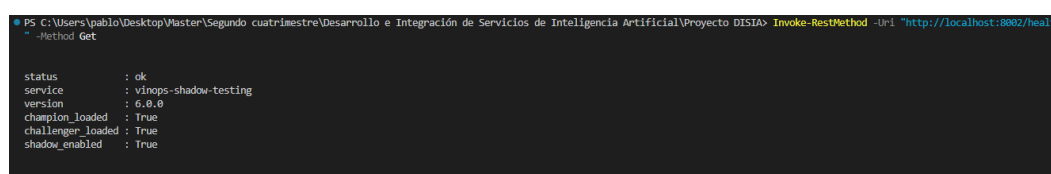
Como extensión opcional del hito, se ha implementado un mecanismo de *shadow testing*, también denominado enfoque *champion-challenger*. En este esquema, el modelo *champion* continúa siendo el único que responde al cliente, mientras que un modelo *challenger* recibe en segundo plano una copia de cada petición con el objetivo de comparar ambos comportamientos sin afectar al servicio principal.

Esta estrategia se ha preferido frente a un A/B testing clásico. La razón es que el A/B testing implicaría repartir el tráfico real entre dos modelos, lo que en un contexto académico y con un servicio todavía en evolución puede introducir riesgos innecesarios sobre la experiencia del consumidor. El shadow testing, en cambio, permite evaluar un candidato sin comprometer la respuesta operacional.

La implementación registra para cada petición la predicción del *champion*, la del *challenger*, la confianza asociada en ambos casos, el grado de acuerdo entre modelos y la latencia observada. Toda esta información se almacena en un CSV de comparaciones que permite inspeccionar posteriormente la consistencia del modelo candidato y su posible idoneidad para promoción.

Cuando el operador decide promover el challenger, el endpoint `/shadow/promote` sustituye el artefacto activo por el modelo candidato y actualiza la instancia cargada en memoria. De este modo, la transición puede realizarse sin reiniciar el contenedor y manteniendo la continuidad operativa del servicio.

Las Figuras 22–25 muestran la validación experimental completa del flujo champion-challenger.



```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA> Invoke-RestMethod -Uri "http://localhost:8002/health" -Method Get
{
  status: ok,
  service: vinops-shadow-testing,
  version: 6.0.0,
  champion_loaded: true,
  challenger_loaded: true,
  shadow_enabled: true
}
```

Figura 22: Respuesta del endpoint `/health` del entorno bonus de shadow testing en el puerto 8002.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA> Invoke-RestMethod -Uri "http://localhost:8082/predict" -Method Post -ContentType "application/json" -Body $body

prediction      : 2
class_label     : Cultivar_2
probabilities   : @(class_1=0,3569; class_2=0,6431; class_3=0,0)
confidence      : 0,6431
model           : champion
shadow_comparison : @(challenger_prediction=2; challenger_confidence=0,6431; agreement=True; confidence_delta=0,0; latency_challenger_ms=29,87)
```

Figura 23: Predicción en modo champion-challenger con comparación entre ambos modelos.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA> Invoke-RestMethod -Uri "http://localhost:8082/shadow/status" -Method Get

shadow_enabled champion_path challenger_path statistics
-----
True /app/models/wine_model.pkl /app/models/wine_model_challenger.pkl @comparisons=20; agreements=20; disagreements=0; agreement_rate=1,0; avg_confidence_champion=0,624; av...
```

Figura 24: Estado agregado del shadow testing con métricas de comparación entre champion y challenger.

```
PS C:\Users\pablo\Desktop\Master\Segundo cuatrimestre\Desarrollo e Integración de Servicios de Inteligencia Artificial\Proyecto DISIA> Invoke-RestMethod -Uri "http://localhost:8082/shadow/promote" -Method Post

status message previous_champion new_champion
-----
promoted Challenger promovido a Champion. El nuevo Champion esta activo. /app/models/wine_model.pkl.backup /app/models/wine_model.pkl
```

Figura 25: Promoción del modelo challenger a champion mediante el endpoint /shadow/promote.

Decisiones de diseño globales

A lo largo del desarrollo del hito se han adoptado varias decisiones con impacto directo sobre la arquitectura y la operatividad del sistema. La Tabla 25 resume las principales elecciones, la alternativa descartada en cada caso y la justificación asociada.

Tabla 25: Decisiones de diseño principales del Hito 5

Decisión	Alternativa descartada	Justificación
Métricas proxy online	F1, precisión o AUC en tiempo real	La etiqueta real no está disponible en el momento de la predicción.
Implementación estadística ligera	Frameworks como Alibi Detect o NannyML	Se prioriza un contenedor más manejable y una solución comprensible en contexto académico.
CSV para feedback	SQLite o PostgreSQL	El volumen esperado es pequeño y el archivo resulta fácil de inspeccionar.
Retraining con originales + feedback duplicado	Reentrenar solo con feedback	Se reduce el riesgo de sobreajuste y de olvido catastrófico en small data.
Trigger de 10 feedbacks	Trigger por eventos de drift	El drift no aporta por sí mismo etiquetas válidas para aprendizaje supervisado.
Telegram como canal de alertas	Correo o plataformas corporativas	La integración es simple y la demostración resulta más visible e inmediata.
Recarga en caliente	Reinicio del contenedor	Se evita downtime y se preserva el estado operativo.
Shadow testing	A/B testing	Permite evaluar modelos candidatos sin afectar al tráfico servido.

Las implicaciones de estas decisiones son ambivalentes. Por un lado, el sistema gana claridad, trazabilidad y facilidad de despliegue en un entorno académico. Por otro, algunas elecciones priorizan la simplicidad frente a la escalabilidad, por lo que en un entorno productivo real sería razonable evolucionar hacia soluciones con versionado de modelos, almacenamiento más robusto y mecanismos automáticos de rollback.

Limitaciones y valoración final del hito

A pesar de los avances alcanzados, la solución implementada presenta varias limitaciones. En primer lugar, el modelo reentrenado sobrescribe el artefacto anterior, por lo que no existe aún un versionado formal ni un mecanismo automático de reversión. En segundo lugar, el feedback utilizado para reentrenar no queda marcado como consumido, lo que podría complicar su trazabilidad si el sistema evolucionara hacia un uso continuado. Finalmente, el entorno de shadow testing se ha planteado como una validación controlada, por lo que su valor demostrativo es mayor que su madurez operativa real.

No obstante, el Hito 5 supone un avance cualitativo importante respecto a los hitos anteriores. El proyecto deja de ser únicamente un modelo desplegado y pasa a incorporar varios elementos esenciales de observabilidad y ciclo de vida MLOps: métricas operativas, métricas proxy del modelo, detección de drift, alertas automáticas, feedback humano, reentrenamiento supervisado y comparación segura entre modelos.

En conjunto, las decisiones adoptadas incrementan la complejidad estructural del sistema respecto al Hito 4, pero lo hacen a cambio de una mejora clara en su capacidad de supervisión, adaptación y explotación. Aunque se trata todavía de una solución local y académica, el resultado aproxima el proyecto a un escenario real de operación y sienta una base razonable para futuras extensiones como MLflow, auto-retraining, canary deployment o estimación avanzada de degradación sin *ground truth*.

Bibliografía

- [ACD91] Aeberhard, Stefan, Danny Coomans y Olivier De Vel (1991). *Wine Recognition Dataset*. UCI Machine Learning Repository. Accedido: febrero 2026. URL: <https://archive.ics.uci.edu/ml/datasets/wine>.
- [ACD94] — (1994). «Comparative Analysis of Statistical Pattern Recognition Methods in High Dimensional Settings». En: *Pattern Recognition* 27.8, págs. 1065-1077. DOI: 10.1016/0031-3203(94)90145-7.
- [Bre01] Breiman, Leo (2001). «Random Forests». En: *Machine Learning* 45.1, págs. 5-32. DOI: 10.1023/A:1010933404324.
- [Cha+02] Chawla, Nitesh V. et al. (2002). «SMOTE: Synthetic Minority Over-sampling Technique». En: *Journal of Artificial Intelligence Research* 16, págs. 321-357. DOI: 10.1613/jair.953.
- [Coh60] Cohen, Jacob (1960). «A Coefficient of Agreement for Nominal Scales». En: *Educational and Psychological Measurement* 20.1, págs. 37-46. DOI: 10.1177/001316446002000104.
- [CV95] Cortes, Corinna y Vladimir Vapnik (1995). «Support-Vector Networks». En: *Machine Learning* 20.3, págs. 273-297. DOI: 10.1007/BF00994018.
- [Fis36] Fisher, Ronald A. (1936). «The Use of Multiple Measurements in Taxonomic Problems». En: *Annals of Eugenics* 7.2, págs. 179-188. DOI: 10.1111/j.1469-1809.1936.tb02137.x.
- [Gar+26a] García, Alejandro et al. (2026). *VinOps: Repositorio del proyecto*. <https://github.com/Pabloalfon/VinOps/>. Repositorio del proyecto que contiene el código y los experimentos.
- [Gar+26b] García Fernández, Alejandro et al. (feb. de 2026). *VinOps: Hito 1 – Definición del problema y determinación del alcance*. Informe de proyecto. Máster en Ingeniería Informática, UCLM.
- [Gla24] Glassdoor (2024). *Salario de Data Scientist en España*. URL: https://www.glassdoor.es/Sueldos/data-scientist-sueldo-SRCH_K00,14.htm.

- [HTF09] Hastie, Trevor, Robert Tibshirani y Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer. DOI: 10.1007/978-0-387-84858-7.
- [Jam+21] James, Gareth et al. (2021). *An Introduction to Statistical Learning with Applications in R*. 2nd. Springer. DOI: 10.1007/978-1-0716-1418-1.
- [LJH08] Lê, Sébastien, Julie Josse y François Husson (2008). «FactoMineR: An R Package for Multivariate Analysis». En: *Journal of Statistical Software* 25.1, págs. 1-18. DOI: 10.18637/jss.v025.i01.
- [Min23] Ministerio de Agricultura, Pesca y Alimentación (2023). *Superficies y producciones de cultivos. Viñedo*. URL: <https://www.mapa.gob.es/es/estadistica/temas/estadisticas-agrarias/agricultura/superficies-producciones-anuales-cultivos/>.
- [Obs22] Observatorio Español del Mercado del Vino (2022). *Informe anual del sector vitivinícola español*. URL: <https://www.oemv.es/>.
- [Org23] Organisation Internationale de la Vigne et du Vin (2023). *State of the World Vine and Wine Sector in 2023*. URL: <https://www.oiv.int/public/medias/9885/oiv-state-of-the-world-vine-and-wine-sector-in-2023.pdf>.
- [R C23] R Core Team (2023). «R: A Language and Environment for Statistical Computing». En: URL: <https://www.R-project.org/>.
- [Wic16] Wickham, Hadley (2016). «ggplot2: Elegant Graphics for Data Analysis». En: *Journal of Statistical Software*. URL: <https://ggplot2.tidyverse.org>.